A Deep High-Order Tensor Sparse Representation for Hyperspectral Image Classification

Chunbo Cheng, Liming Zhang, *Senior Member, IEEE*, Hong Li, *Member, IEEE*, Wenjing Cui, Junbin Gao, *Member, IEEE*, and Yuxiao Cun

***Abstract*— Deep learning-based hyperspectral image (HSI) classification methods have recently shown excellent performance. However, the success of these deep learning methods mainly relies on the deep network architecture with a huge amount of parameters trained by a large number of training samples. In this article, a deep high-order tensor sparse representation (SR) network (DHTSRNet) is proposed, which can obtain better classification results in the case of small training samples. Specifically, we propose a high-order tensor SR (HTSR) model that can handle arbitrary-order tensor-type data, and extend it to a deep HTSR model that can be used to train deep high-order tensor filters and features. Then, a deep feature extraction network (DHTSRNet) based on the deep HTSR model is constructed, which is used for feature extraction of HSI. Finally, an HSI classification method is constructed by combining DHTSRNet and the classifier based on graph-based learning (GSL), which can obtain better classification results in the case of small training samples. Experimental results show that the DHTSRNet can obtain better classification performance compared with other state-of-the-art HSI classification methods.**

***Index Terms*— Convolutional neural network (CNN), deep high-order tensor sparse representation (SR), deep learning, graph-based learning (GSL), hyperspectral image (HSI) classification.**

I. Introduction

HYPERSPECTRAL image (HSI) classification requires labeling each pixel in the imagery as belonging to one of the predefined land cover categories [1]. It has been applied widely in fields such as geological and mineral resources exploration, crop growth and yield assessment, ecological protection and environmental monitoring of mines, water quality testing, and soil monitoring [2]. Therefore, HSI classification has become a highly focused research area. Because the features of HSIs usually contain a lot of redundant or irrelevant information, feature selection or dimensionality reduction is required before classification. Correct selection of relevant features and removing irrelevant features is crucial to improve classification performance [3]. The principal component analysis (PCA) [4] is a classical method for dimension reduction. In order to retain spatial–spectral information useful for classification, Xia et al. [5] proposed a rotating random forest (RoRF-KPCA) for nuclear PCA. Wu et al. [6] proposed an unsupervised band selection method based on the heterogeneous Cuckoo search algorithm with matched filter (HCSMF) for HSI classification, which is based on edge-preserving filtering-based support vector machine (EPF-SVM) classifiers. These methods mainly focus on spectral feature extraction. Due to the complex spatial distribution and spectral heterogeneity of the objects, classification only relying on spectral features usually cannot obtain high classification accuracy [7]. Jia et al. [8] proposed a 3-D local binary pattern (3D-LBP) model by forming a 3-D regular octahedral frame to characterize the spectral–spatial relationship. Gao et al. [9] proposed a spectral and spatial classification framework for HSI, which uses random multigraphs (RMGs). Liu et al. [10] presented a sparse representation (SR) classification framework based on a spatial–spectral kernel. Although the above methods have achieved good results, these methods suffer from the disadvantages of low generalizability and limited representational capacity.

In recent years, researchers have introduced deep learning methods [1] for automated learning from HSI to extract features suitable for classification tasks [2]. Among the several deep-learning models, convolutional neural networks (CNNs) are the most widely used. Miclea et al. [11] proposed 1D-CNN, which outperforms traditional classifiers such as SVM and random forest (RF). Lee and Kwon [12] designed 2D-CNN and 3D-CNN capable of utilizing joint-spectral–spatial information in order to improve HSI classification, with 2D-CNN using different bands of HSIs as feature maps and 3D-CNN using HSI data cubes directly as input. Cheng et al. presented a deep high-order tensor convolutional sparse coding

Manuscript received 12 March 2024; revised 14 May 2024 and 7 June 2024; accepted 20 June 2024. Date of publication 25 June 2024; date of current version 5 July 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62206088, in part by the Talent Introduction Program from Hubei Polytechnic University under Grant 21xjz25R, in part by the Science and Technology Development Fund of Macau under Grant 0041/2023/RIA1, in part by the Multiyear Research Grant MYRG2022-00193-FST and Grant MYRG-GRG2023-00056-FST-UMDF, and in part by the Yunnan Provincial Department of Education Science Research Fund under Project 2023J1275. *(Corresponding author: Liming Zhang.)*

Chunbo Cheng and Wenjing Cui are with the School of Mathematics and Physics, Hubei Key Laboratory of Intelligent Convey Technology and Device, Hubei Polytechnic University, Huangshi, Hubei 435000, China (e-mail: bccheng@hbpu.edu.cn; wjcui@hbpu.edu.cn).

Liming Zhang is with the Faculty of Science and Technology, University of Macau, Macau, China (e-mail: lmzhang@umac.mo).

Hong Li is with the School of Mathematics and Statistics, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: hongli@hust.edu.cn).

Junbin Gao is with the Discipline of Business Analytics, The University of Sydney Business School, The University of Sydney, Camperdown, NSW 2006, Australia (e-mail: junbin.gao@sydney.edu.au).

Yuxiao Cun is with the Public Basic Course Teaching Department, West Yunnan University of Applied Sciences, Lincang, Yunnan 671002, China (e-mail: 1170060@wyuas.edu.cn).

Digital Object Identifier 10.1109/TGRS.2024.3418785

1558-0644 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 04:15:19 UTC from IEEE Xplore. Restrictions apply.

(CSC) model-based CNN, which first uses high-order CSC to train the parameters of CNN, and then extracts the features of HSI and uses graph-based learning (GSL) method [7], [9], [13] to classify it. Wang et al. [14] proposed an end-to-end dense spectral–spatial convolution (FDSSC) framework, which extracts the spectral and spatial features of HSI. Ben Hamida et al. [15] proposed a 3D-CNN that combines spectral and spatial information for HSI classification. Song et al. [16] introduced residual learning to increase the depth of the network. Li et al. [17] proposed a two-stream 2-D CNN structure, which separately extracts global features and local features by the two-stream network. Cheng et al. [18] proposed a deep network architecture (SAFDNet) based on the stochastic adaptive Fourier decomposition (SAFD) theory, and the network is based on CNN. Roy et al. [19] proposed a hybrid 2D–3D CNN for feature extraction. Zhong et al. [20] proposed a residual network based on spectral and spatial features.

In addition, based on the self-attention mechanism, transformer [21] has been applied in HSI classification and become more and more successful. By combining the attention mechanism with CNN, an attention mechanism-based method termed the multilevel feature network with spectral–spatial attention model (MFNSAM) was proposed [22]. Liu et al. [23] proposed a central attention network (CAN), which combined HSI-tailored dense connections of the features of the hidden layers and the spectral information of the query pixel. In [24], a multiscale model called multiarea target attention (MATA) was proposed, which used an architecture that includes a shared feature extractor (FE) and classifier to capture multiscale spectral–spatial information effectively and efficiently. Ye et al. [25] proposed a framework called adaptive domain-adversarial few-shot learning (ADAFSL), which employed an adaptive strategy that assigned varying weights to the conditional adversarial losses for different classes based on their respective degrees of discrepancies, thereby achieving global conditional distribution alignment. Zhang et al. [26] proposed a structural optimization transmission framework, namely, structural optimization transmission network (SOT-Net), for collaborative land-cover classification of HSI and LiDAR data. A spatial-logical aggregation network (SLA-NET) with morphological transformation for tree species classification was proposed in [27], which focused on subtle traits extraction. Zhang et al. [28] proposed a multilevel DA network (MDA-NET) for promoting information collaboration and cross-scene (CS) classification based on HSI and light detection and ranging (LiDAR) data.

The reason why CNN has achieved such great success is that it can extract effective deep features with strong robustness and invariance [7] compared with shallow features. However, HSIs usually have unlabeled or sparsely labeled samples, while deep networks reliant on data generally demand a substantial amount of training information [29]. Therefore, the prominent issue of small sample size in hyperspectral remote sensing images becomes a crucial factor limiting the further enhancement of deep learning for HSI classification performance [29]. Simultaneously, labeling HSIs is a time-consuming, labor-intensive, and expensive process [29]. It is necessary to design a more discriminative feature extraction method. Therefore, how to extract fruitful features from such high-dimensional limited data efficiently and effectively has become a critical problem [30].

SR is an extremely effective method and commonly used feature extraction for exploiting the intrinsic structure of high-dimensional data [31], [32], [33], [34]. The goal of SR is to find a compact representation of high-dimensional data by selecting a small subset of a given dictionary to identify low-dimensional structures in high-dimensional data [31]. High-dimensional data are often characterized by intrinsically low-dimensional structures [31]. Consequently, exploiting the low-dimensional structures in high-dimensional data has received considerable attention [35], [36]. The dictionary and SR coefficients learned by the SR model can represent the intrinsic features of data. Specifically, SR coefficients can be employed to obtain the weights for the pairwise relationship graph, which measures the similarity among high-dimensional samples [31], [37]. The elements in the dictionary can be seen as a collection of filters, with different filters representing different functions. Different filters can extract different features of the image, such as using different filters to extract specific information including contour or color and so on. Most existing SR models can only handle two-way (matrix) data. However, HSI is usually 3-D data (multidimensional) in nature, and the information is stored in multiway arrays known as tensors [38]. To be able to handle high-dimensional data directly, multiway data have to be reconstructed into matrices first [32], [33], [34]. Such a preprocessing usually leads to an information loss and would cause performance degradation.

These methods are more suitable to deal with low-order tensor data and are not generalized well to any order tensor data. In addition, feature extraction based on tensor [39] or matrix can only extract shallow features, and few deep learning methods directly deal with higher-order tensor-type data at present. This makes it difficult for these methods to be used to train the parameters of deep networks. The success of CNN methods mainly relies on the deep network architecture with a huge amount of parameters trained by a large number of training samples. It has been found that the network parameters have been substituted by the weight matrices or tensors by training tensor convolutional sparse coding model [7]. The deep network parameters (such as CNNs) are usually 4-D tensors. Nevertheless, these DL methods will fold tensors into matrices for training, and fail to utilize the tensor-based network structure [40]. In addition, when tensor data is folded into a matrix, the spatial characteristics of the data are easily lost. In order to make full use of the spatial structure of the data and extract the deep features of the data, it is necessary to propose a deep higher-order tensor model.

Recently, tensor-based decomposition methods have achieved significant performance in many applications [41], [42], [43]. In the HSI-SR problem, they have advantages preserving the spatial–spectral structure of high-resolution HSI (HR-HSI) [43]. The common tensor decomposition methods are canonical polyadic decomposition [44], Tucker

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 04:15:19 UTC from IEEE Xplore. Restrictions apply.

decomposition [45], the tensor train decomposition [46], and the tensor ring decomposition [47]. However, these methods still have some weaknesses. On the one hand, some deep structural information like multiscale correlation is not considered [43]. On the other hand, these decompositions only establish a connection between adjacent two factors and thus are highly sensitive to the permutation of tensor modes [43].

In this article, we aim to propose an SR model of arbitrary order tensor and generalize it to the deep model, which can handle both higher-order tensor-based data and extract deep features. Based on a deep high-order tensor SR (HTSR) model, a deep feature extraction network (DHTSRNet) is constructed, which can be used to extract deep features. Combining the classifier GSL, an effective HSI classification method is constructed, which can obtain high classification accuracy in the case of small training samples.

The main contributions and novelty of this article are summarized as follows.

1) A HTSR model is proposed, which can handle arbitrary order tensor-type data, while traditional SR models can only handle lower order tensor-type data.
2) We generalize the higher-order tensor SR model to the deep model and propose a deep HTSR model, which can be used to train deep high-order tensor filters and features, while conventional SR models can only learn shallow features and shallow filters.
3) Based on the deep HTSR model, a deep feature extraction network (DHTSRNet) is constructed, which is used for feature extraction of HSIs. GSL is used to classify the extracted deep features. An HSI classification method is constructed by combining the deep HTSR model and the classifier GSL, which can obtain high classification accuracy in the case of small training samples.

The remainder of this article is organized as follows. Section II introduces notations and the preliminary knowledge of tensors. Section III presents the proposed HTSR model, the optimization and convergence analysis are also presented in this section. In addition, Section III also shows a deep higher-order SR network for HSIs, the classification framework, and the complexity analysis of the whole classification method. Sections IV and V are experiments and conclusions, respectively.

II. Notations and Preliminaries

A. Notations

In this section, we introduce the notations that will be used throughout the article. Scalars are tensors of order zero, vectors are tensors of order one, and matrices are tensors of order two, which will be denoted by lowercase, bold lowercase, and bold capital letters (e.g., a, $\mathbf{a}$, $\mathbf{A}$), respectively. Higher order tensors refer to tensors of order three and more, and they will be denoted by boldface Euler script letters (e.g., $\mathcal{A}$). For an dth order tensor $\mathcal{A} \in \mathbb{R}^{n_1 \times n_2 \times \cdots \times n_d}$, The Frobenius squared norm is $\|\mathcal{A}\|_F^2 = \sum_{i_1,i_2,\ldots,i_d} \mathcal{A}(i_1, i_2, \ldots, i_d)^2$, while its $\ell_{1,1,\ldots,1}$ norm is $\|\mathcal{A}\|_{1,1,\ldots,1} = \sum_{i_1,i_2,\ldots,i_d} |\mathcal{A}(i_1, i_2, \ldots, i_d)|$. The inner product between two tensors of the same size is $\langle \mathcal{A}, \mathcal{B} \rangle = \sum_{i_1,i_2,\ldots,i_d} \mathcal{A}(i_1, i_2, \ldots, i_d)\mathcal{B}(i_1, i_2, \ldots, i_d)$. Usually, a tensor is unfolded into a matrix. The unfold operation can be done along any of its dimensions. A k-mode fold/unfold of a tensor is defined as $\text{unfold}_k(\mathcal{A}) = \mathbf{A}_{(k)} \in \mathbb{R}^{n_k \times (n_1 n_2 \cdots n_{k-1} n_{k+1} \cdots n_d)}$ and $\text{fold}_k(\mathbf{A}_{(k)}) = \mathcal{A}$. A tensor product depends on the t-product theory. The dth order t-product establishes the existing literature [48].

Definition 1 (High Order Block Circulant Matrix [48]):

$$\text{circ}(\mathcal{A}) = \begin{bmatrix} \mathbf{A}^{(1)} & \mathbf{A}^{(n_d)} & \mathbf{A}^{(n_d-1)} & \cdots & \mathbf{A}^{(2)} \\ \mathbf{A}^{(2)} & \mathbf{A}^{(1)} & \mathbf{A}^{(n_d)} & \cdots & \mathbf{A}^{(3)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \mathbf{A}^{(n_d)} & \mathbf{A}^{(n_d-1)} & \mathbf{A}^{(n_d-2)} & \cdots & \mathbf{A}^{(1)} \end{bmatrix} \quad (1)$$

where the notation $\text{bcirc}(\mathcal{A}) \in \{n_1 n_3 \cdots n_d \times n_2 n_3 \cdots n_d\}$ is a block circulant matrix at the base level of the operator $\text{circ}(\mathcal{A}) \in \mathbb{R}^{n_1 n_d \times \cdots \times n_{d-2} n_d \times n_{d-1}}$, and $\mathbf{A}^{(i)} = \mathcal{A}(:, \ldots, :, i) \in \mathbb{R}^{n_1 \times \cdots \times n_{d-1}}$ is the $(d-1)$th order tensor constructed by keeping the dth index of $\mathcal{A}$ fixed at i. The notation $\text{bdiag}(\mathcal{A}) = \text{diag}(\mathbf{B}^{(1)}, \mathbf{B}^{(2)}, \ldots, \mathbf{B}^{(D-1)}, \mathbf{B}^{(D)}) \in \mathbb{R}^{n_1 n_3 \cdots n_d \times n_2 n_3, \ldots, n_d}$ is a block diagonal matrix, $\mathbf{B}^{(j)} = \mathcal{A}(:, :, i_3, \ldots, i_d) \in \mathbb{R}^{n_1 \times n_2}$, $j = (i_d - 1)n_3 \cdots n_{d-1} + \cdots + (i_4 - 1)n_3 + i_3$, $i_d \in \{1, 2, \ldots, n_d\}$ and $D = n_3, \ldots, n_d$.

Definition 2 (Tensor Vectorization [48]):

$$\text{unfold}(\mathcal{A}) = \begin{bmatrix} \mathbf{A}^{(1)} \\ \mathbf{A}^{(2)} \\ \vdots \\ \mathbf{A}^{(n_d)} \end{bmatrix}, \quad \text{fold}(\text{unfold}(\mathcal{A})) = \mathcal{A}. \quad (2)$$

Definition 3 (High Order t-Products [48]): For any $\mathcal{W} \in \mathbb{R}^{n_1 \times K \times n_2 \times \cdots \times n_d}$

$$\mathcal{W} \otimes_{\text{HO}} \mathcal{X} = \text{fold}(\text{bcirc}(\mathcal{W})\text{unfold}(\mathcal{X})). \quad (3)$$

B. Preliminaries

Based on the above t-product Definition 3, SR can be written as a linear sum of t-products, if the images/filters are 1-D signals (i.e., vectorized patches)

$$\min_{\mathcal{W},\mathcal{V}} \frac{1}{2} \sum_{m=1}^{M} \|\mathcal{H}_m - \mathcal{W} \otimes \mathcal{V}_m\|_F^2 + \lambda \|\mathcal{V}_m\|_{1,1,1} \quad (4)$$

where $\otimes$ is defined as the t-product in (3) when $n_i = 1$ $(i \geq 3)$, $\mathcal{H}_m \in \mathbb{R}^{n_1 \times 1 \times n_2}$, $\mathcal{W} \in \mathbb{R}^{n_1 \times K \times n_2}$, and $\mathcal{V}_m \in \mathbb{R}^{K \times 1 \times n_2}$. λ is a regularization parameter. The t-product reduces to the standard matrix multiplication when $n_i = 1$ $(i \geq 3)$.

III. Deep HTSR for HSI Classification

The traditional SR model ignores the 2-D spatial structure in images and cannot handle multidimensional features at each pixel. To do this, we first extend the SR model to an arbitrary-order tensor model, which can handle high-order tensor or multidimensional data. Second, the great success of deep learning is due to its powerful ability to represent complex and deep features. However, there are currently few HTSR models used to extract deep features in the literature. In this

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 04:15:19 UTC from IEEE Xplore. Restrictions apply.

article, we extend the proposed higher-order tensor SR model to a deep higher-order SR network. Finally, we present the application of the proposed HTSR model to HSI and establish an HSI classification algorithm.

A. HTSR Model

The traditional SR model ignores the 2-D spatial structure in images and cannot handle multidimensional features at each pixel. To overcome the issues of previous tensor formulations in (4), we aim to construct an arbitrary order tensor SR model on the basis of the traditional SR model in (4). For the multidimensional image $\mathcal{T}_m \in \mathbb{R}^{n_1\times 1\times n_2\times\cdots\times n_d}$, the dictionary $\mathcal{Q} \in \mathbb{R}^{n_1\times K\times n_2\times\cdots\times n_d}$ and the sparse code $\mathcal{U}_m \in \mathbb{R}^{K\times 1\times n_2\times\cdots\times n_d}$, the tensor SR model for arbitrary dth order tensor can be formulated as

$$\min_{\mathcal{Q},\mathcal{U}} \frac{1}{2}\sum_{m=1}^{M}\|\mathcal{T}_m - \mathcal{Q}\otimes_{\mathrm{HO}}\mathcal{U}_m\|_F^2 + \lambda\|\mathcal{U}_m\|\underbrace{1,1,\ldots,1}_{d+1} \tag{5}$$

where $\otimes_{\mathrm{HO}}$ is the high order t-product, as defined in (3). When $n_i = 1\ \forall i \geq 3$, (5) reduces back to standard SR model in (4).

The t-product is analogous to the matrix multiplication except that the circular convolution replaces the multiplication operation between the elements. Since the t-product is equivalent to the matrix multiplication in the Fourier domain, for computational efficiency, we convert the objective (5) into the Fourier domain by using the definition of the operator $\otimes_{\mathrm{HO}}$, which can be obtained by the following definition.

Definition 4 (Block Diagonalization Property): For any $\mathcal{W} \in \mathbb{R}^{n_1\times K\times n_2\times\cdots\times n_d}$

$$\begin{aligned}\hat{\boldsymbol{W}} = &\left(\boldsymbol{F}_{n_d}\otimes\boldsymbol{F}_{n_{d-1}}\otimes\cdots\otimes\boldsymbol{F}_{n_2}\otimes\boldsymbol{I}_{n_1}\right)\mathrm{bcirc}(\mathcal{W})\\ &\times\left(\boldsymbol{F}_{n_d}^{-1}\otimes\boldsymbol{F}_{n_{d-1}}^{-1}\otimes\cdots\otimes\boldsymbol{F}_{n_2}^{-1}\otimes\boldsymbol{I}_K\right)\end{aligned} \tag{6}$$

where $\boldsymbol{F}_i$ $(i \in \{n_1, n_2, \ldots, n_d\})$ is the discrete Fourier transformation (DFT) matrix, and $\boldsymbol{F}_i\boldsymbol{F}_i^* = \boldsymbol{F}_i^*\boldsymbol{F}_i = i\boldsymbol{I}_i$, thus $\boldsymbol{F}_i^{-1} = \boldsymbol{F}_i^*/i$. $\boldsymbol{I}_i$, $\boldsymbol{I}_{n_1}$, and $\boldsymbol{I}_K$ are the identity matrix. $\hat{\boldsymbol{W}} = \mathrm{bdiag}(\hat{\mathcal{W}})$, and we denote $\hat{\mathcal{W}}$ as the result of DFT on $\mathcal{W}$ along dimensions $(3, 4, \ldots, d)$.

Definition 5 (Equivalent Property): For any $\mathcal{W} \in \mathbb{R}^{n_1\times K\times n_2\times\cdots\times n_d}$

$$\mathcal{W}\otimes_{\mathrm{HO}}\mathcal{X} \Longleftrightarrow \hat{\boldsymbol{W}}\hat{\boldsymbol{X}} \tag{7}$$

where $\hat{\boldsymbol{X}} = \mathrm{bdiag}(\hat{\mathcal{X}})$, and we denote $\hat{\mathcal{X}}$ as the result of DFT on $\mathcal{X}$ along dimensions $(3, 4, \ldots, d)$.

Indeed, $\mathcal{P} = \mathcal{W}\otimes_{\mathrm{HO}}\mathcal{X}$ is equivalent to $\hat{\boldsymbol{P}} = \hat{\boldsymbol{W}}\hat{\boldsymbol{X}}$ in the Fourier domain, which is derived from the following formula by Definitions 2–4:

$$\begin{aligned}&\mathrm{unfold}(\mathcal{P})\\ &= \mathrm{bcirc}(\mathcal{W})\mathrm{unfold}(\mathcal{X})\\ &= \left(\boldsymbol{F}_{n_d}^{-1}\otimes\boldsymbol{F}_{n_{d-1}}^{-1}\otimes\cdots\otimes\boldsymbol{F}_{n_2}^{-1}\otimes\boldsymbol{I}_{n_1}\right)\\ &\quad\cdot\left(\left(\boldsymbol{F}_{n_d}\otimes\boldsymbol{F}_{n_{d-1}}\otimes\cdots\otimes\boldsymbol{F}_{n_2}\otimes\boldsymbol{I}_{n_1}\right)\cdot\mathrm{bcirc}(\mathcal{W})\right.\\ &\qquad\left.\cdot\left(\boldsymbol{F}_{n_d}^{-1}\otimes\boldsymbol{F}_{n_{d-1}}^{-1}\otimes\cdots\otimes\boldsymbol{F}_{n_2}^{-1}\otimes\boldsymbol{I}_K\right)\right)\\ &\quad\cdot\left(\left(\boldsymbol{F}_{n_d}\otimes\boldsymbol{F}_{n_{d-1}}\otimes\cdots\otimes\boldsymbol{F}_{n_2}\otimes\boldsymbol{I}_K\right)\cdot\mathrm{unfold}(\mathcal{X})\right)\\ &= \left(\boldsymbol{F}_{n_d}^{-1}\otimes\boldsymbol{F}_{n_{d-1}}^{-1}\otimes\cdots\otimes\boldsymbol{F}_{n_2}^{-1}\otimes\boldsymbol{I}_{n_1}\right)\cdot\hat{\boldsymbol{W}}\cdot\mathrm{unfold}\left(\hat{\boldsymbol{X}}\right).\end{aligned} \tag{8}$$

Left multiplying both sides in (8) with $\boldsymbol{F}_{n_d}\otimes\boldsymbol{F}_{n_{d-1}}\otimes\cdots\otimes\boldsymbol{F}_{n_2}\otimes\boldsymbol{I}_{n_1}$ leads to $\mathrm{unfold}(\hat{\mathcal{P}}) = \hat{\boldsymbol{W}}\cdot\mathrm{unfold}(\hat{\mathcal{X}})$, this is equivalent to $\hat{\boldsymbol{P}} = \hat{\boldsymbol{W}}\hat{\boldsymbol{X}}$. This property indicates that calculating t-product based on fast Fourier transform (FFT) is an efficient calculation method, rather than using (3) directly.

B. Optimization

To solve (5), we use the traditional fixed point strategy to alternate between the optimization of the SR $\mathcal{U}$ and the dictionary $\mathcal{Q}$. Specifically, (5) can be divided into two subproblems for solving, namely, the SR $\mathcal{U}$ in (5) is solved by fixing the dictionary $\mathcal{Q}$, and the dictionary $\mathcal{Q}$ in (5) is solved by fixing the SR $\mathcal{U}$. Since each subproblem is convex, the alternating direction method of multipliers (ADMMs) can be used to solve it.

According to Definition 5, all computations involving the operator $\otimes_{\mathrm{HO}}$ in the process of solving the variables $\mathcal{U}$ and $\mathcal{Q}$ are converted into the Fourier domain.

Subproblem 1 (SR): Fixing the dictionary $\mathcal{Q}$, then (5) contains only one variable to be solved, which becomes a convex optimization problem as follows:

$$\begin{aligned}&\operatorname*{argmin}_{\mathcal{Q},\mathcal{R}} \frac{1}{2}\sum_{m=1}^{M}\|\mathcal{T}_m - \mathcal{Q}\otimes_{\mathrm{HO}}\mathcal{U}_m\|_F^2 + \lambda\|\mathcal{R}_m\|\underbrace{1,1,\ldots,1}_{d+1}\\ &\text{s.t. } \mathcal{U}_m = \mathcal{R}_m \quad \forall m = 1, 2, \ldots, M.\end{aligned} \tag{9}$$

By introducing auxiliary variable $\mathcal{R}$, (5) is split into the smooth and nonsmooth parts, and ADMM can be used to iteratively minimize the primal variables $(\mathcal{U}, \mathcal{R})$ and the dual variable B. The augmented lagrangian is given as follows:

$$\begin{aligned}&\mathcal{L}(\mathcal{U}_m, \mathcal{R}_m, \mathcal{B}_m)\\ &= \frac{1}{2}\sum_{m=1}^{M}\|\mathcal{T}_m - \mathcal{Q}\otimes_{\mathrm{HO}}\mathcal{U}_m\|_F^2\\ &\quad+\lambda\left\|\mathcal{R}_m\right\|\underbrace{1,1,\ldots,1}_{d+1} + \frac{\beta}{2}\sum_{m=1}^{M}\left\|\mathcal{U}_m - \mathcal{R}_m\right\|_F^2\\ &\quad+\sum_{m=1}^{M}\langle\mathcal{B}_m, (\mathcal{U}_m - \mathcal{R}_m)\rangle \quad \forall m = 1, 2, \ldots, M.\end{aligned} \tag{10}$$

Step 1 (Update $\mathcal{U}$):

$$\mathcal{U}_m^i \longleftarrow \mathrm{ifft}\left(\left(\left(\hat{\mathcal{Q}}^i\right)^T\hat{\mathcal{Q}}^i + \beta\mathbf{I}_K\right)^{-1}\left(\left(\hat{\mathcal{Q}}^i\right)^T\hat{\mathcal{T}}_m^i + \beta\hat{\mathcal{R}}_m^i - \hat{\mathcal{B}}_m^i\right)\right) \tag{11}$$

the ith dimension $\mathcal{U}_m^i(\forall m = 1, 2, \ldots, M)$ is concatenated along the second dimension as $\mathcal{U} \in \mathbb{C}^{K\times M\times n_2\times\cdots\times n_d}$. "ifft" stands for inverse Fourier transform.

Step 2 (Update $\mathcal{R}$ and $\mathcal{B}$):

$$\begin{aligned}\mathcal{R}_m &\longleftarrow \operatorname*{argmin}_{\mathcal{R}_m}\lambda\|\mathcal{R}_m\|\underbrace{1,1,\ldots,1}_{d+1} + \frac{\beta}{2}\|\mathcal{R}_m - \mathcal{Z}_m\|_F^2\\ \mathcal{Z}_m &= \mathcal{U}_m + \frac{1}{\beta}\mathcal{B}_m\\ \mathcal{B}_m &\longleftarrow \mathcal{B}_m + \beta(\mathcal{U}_m - \mathcal{R}_m) \quad \forall m = 1, 2, \ldots, M.\end{aligned} \tag{12}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 04:15:19 UTC from IEEE Xplore. Restrictions apply.

The soft thresholding operator $\mathcal{S}(x) = \text{sign}(x)\max(0, |x| - (\lambda/\beta))$ is the proximal operator to the ℓ_1 norm, which is applied in an element-wise fashion to tensor $\mathcal{Z}_m$.

Subproblem 2 (Dictionary): Fixing the SR $\mathcal{U}$, then (5) contains only one variable $\mathcal{Q}$ to be solved, which becomes a convex optimization problem with an unconstrained condition

$$\min_{\mathcal{Q}} \frac{1}{2}\sum_{m=1}^{M}\|\mathcal{T} - \mathcal{Q}\otimes_{\text{HO}}\mathcal{U}_m\|_F^2. \tag{13}$$

In this case, we need to add an indicative function to rewrite the original target problem, where $\varphi(\mathcal{Q}_k)$ is an indicative function of $\mathcal{Q}_k$, which makes it easy to use ADMM computing framework

$$\begin{aligned}&\min_{\mathcal{Q}} \frac{1}{2}\sum_{m=1}^{M}\|\mathcal{T} - \mathcal{Q}\otimes_{\text{HO}}\mathcal{U}_m\|_F^2 + \sum_{k=1}^{K}\varphi(\mathcal{Q}_k)\\ &\text{s.t. } \|\mathcal{Q}_k\|_F^2 \le 1 \quad \forall k = 1, 2, \ldots, K.\end{aligned} \tag{14}$$

The augmented Lagrangian is

$$\begin{aligned}\mathcal{L}(\mathcal{Q}, \mathcal{J}, \mathcal{S}) = &\frac{1}{2}\sum_{m=1}^{M}\|\mathcal{T}_m - \mathcal{Q}\otimes_{\text{HO}}\mathcal{U}_m\|_F^2 + \frac{\eta}{2}\|\mathcal{Q} - \mathcal{J}\|_F^2\\ &+ \langle\mathcal{S}, \mathcal{Q} - \mathcal{J}\rangle + \sum_{k=1}^{K}\varphi(\mathcal{J}_k).\end{aligned} \tag{15}$$

Step 3 (Update $\mathcal{Q}$):

$$\begin{aligned}\mathcal{Q}^i \longleftarrow \text{ifft}\Bigg(&\left(\left(\sum_{m=1}^{M}\hat{\mathcal{T}}_m^i\left(\hat{\mathcal{U}}_m^i\right)^H\right) + \eta\hat{\mathcal{J}}^i - \hat{\mathcal{S}}^i\right)\\ &\times\left(\left(\sum_{m=1}^{M}\hat{\mathcal{U}}_m^i\left(\hat{\mathcal{U}}_m^i\right)^H\right) + \eta\mathbf{I}_K\right)^{-1}\Bigg).\end{aligned} \tag{16}$$

This is solved again for a linear index $i = 1, 2, \ldots, n_2n_3, \ldots, n_d$.

Step 4 (Update $\mathcal{J}$ and $\mathcal{S}$):

$$\begin{aligned}\mathcal{J} &\longleftarrow \underset{\mathcal{J}}{\text{argmin}}\,\mathcal{L}(\mathcal{Q}, \mathcal{J}, \mathcal{S})\\ \mathcal{S} &\longleftarrow \mathcal{S} + \eta(\mathcal{Q} - \mathcal{J}).\end{aligned} \tag{17}$$

C. Convergence Analysis

Based on [49], we can deduce that the solution algorithm in Section III-B is convergent. Specifically, using Lemmas 1 and 2 in [49], Theorem 1 can be derived. Theorem 1 shows that the proposed algorithm is convergent.

Lemma 1: $f(\mathcal{Q}, \mathcal{U}) = (1/2)\sum_{m=1}^{M}\|\mathcal{T}_m - \mathcal{Q}\otimes_{\text{HO}}\mathcal{U}_m\|_F^2$ is a semi-algebraic.

Lemma 2: (a) Let $x \in \mathbb{R}^n$ for some $n \in \mathbb{N}$ then the sparse inducing function, $\Phi(x) = \|x\|_0$ is semi-algebraic. (b) Let π be a semi-algebraic set, then the indicator function Ψ_π is semi-algebraic.

Condition 1 (Conditions for the Convergence of ADMM):

1) $\beta > 0$, and for all x, there exist ϱ_1 and ϱ_2, such that $\varrho_1 \succeq \nabla^2 f(x) \succeq \varrho_2$.
2) When (a) holds, there exists some $\varepsilon > 0$, such that $\varrho_2 \succeq (\varepsilon - \beta)\mathbf{I}$ and $(\beta\varepsilon/2)\mathbf{I} \succ [\nabla^2 f(x)]^2$ for all x.
3) There exists a cluster point, denoted by (x^*, z^*, y^*) in $\{(x^t, z^t, y^t)\}$, the sequence generated by ADMM.
4) f and g are semi-algebraic functions, where $\varrho_1 \succeq \nabla^2 f(x)$ means that the matrix $(\varrho_1 - \nabla^2 f(x))$ is positive semi-definite, and $(\beta\varepsilon/2)\mathbf{I} \succ [\nabla^2 f(x)]^2$ means that the matrix $((\beta\varepsilon/2)\mathbf{I} - [\nabla^2 f(x)]^2)$ is positive definite.

From Lemmas 1 and 2 and Condition 1, we can conclude that the proposed optimization algorithm is convergent.

Theorem 1: The sequence $(\mathcal{Q}^t, \mathcal{U}_m^t, \mathcal{J}^t, \mathcal{R}_m^t, \mathcal{S}^t, \mathcal{B}_m^t)$ generated by ADMM, If (a), (b), (c), and (d) of Condition 1 hold, then sequence $\{(\mathcal{Q}^t, \mathcal{U}_m^t, \mathcal{J}^t, \mathcal{R}_m^t, \mathcal{S}^t, \mathcal{B}_m^t)\}$ converges globally to $(\mathcal{Q}^*, \mathcal{U}_m^*, \mathcal{J}^*, \mathcal{R}_m^*, \mathcal{S}^*, \mathcal{B}_m^*)$ and the sequence is of finite length

$$\sum_{t=1}^{\infty}\left\|\psi^{t+1} - \psi^t\right\| < \infty, \quad \psi = (\mathcal{Q}, \mathcal{U}_m) \quad \forall m = 1, 2, \ldots, M \tag{18}$$

where t is the number of iterations.

Proof: For the nonsmooth nonconvex proximal splitting framework is meant to resolve problems that take the following form:

$$\underset{x}{\text{argmin}}\, f(x) + g(x) \tag{19}$$

where f is a nonconvex C^1 function, the gradient ∇f of the function f is Lipschitz continuous, and g is a nonconvex, nonsmooth, and proper closed function. The above formula can be solved using the ADMM algorithm, and the convergence of this algorithm has been proved in literature [49], which satisfies Lemmas 1 and 2 and Condition 1. Obviously, if the proposed optimization algorithm can be placed under this framework, its convergence can be easily proved. The key is how to construct f and g. Once f and g are determined, the proposed algorithm satisfies the conditions (a), (b), (c), and (d) in Condition 1, which indicates that the proposed algorithm is convergent.

To cast the proposed optimization problem within this framework, we treat $\mathcal{Q}$ and $\mathcal{U}_m$ as optimization variables by letting $x = (\mathcal{Q}, \mathcal{U}_m)$, auxiliary variable $z = (\mathcal{J}, \mathcal{R}_m)$, and $y = (\mathcal{S}, \mathcal{B}_m)$, then define

$$f(\mathcal{Q}, \mathcal{U}_m) = \frac{1}{2}\sum_{m=1}^{M}\|\mathcal{T}_m - \mathcal{Q}\otimes_{\text{HO}}\mathcal{U}_m\|_F^2 \tag{20}$$

and additional constraints are formulated as a function

$$g(\mathcal{Q}, \mathcal{U}_m) = \lambda\sum_{m=1}^{M}\|\mathcal{U}_m\|_{\underbrace{1, 1, \ldots, 1}_{d+1}} + \alpha\varphi(\mathcal{Q}) \tag{21}$$

where $\varphi(\mathcal{Q})$ is an indicative function of $\mathcal{Q}$. An optimization algorithm to achieve convergence in solving (19) requires that functions f, g, and $f + g$ are Kurdyka–Łojasiewicz (KL) functions [49]. Apparently, according to Lemma 1, the constructed $f(\mathcal{Q}, \mathcal{U}_m)$ is a semi-algebra. As for $g(\mathcal{Q}, \mathcal{U}_m)$, since $\varphi(\mathcal{Q})$ is indicative function, $\|\mathcal{U}_m\|_{\underbrace{1, 1, \ldots, 1}_{d+1}}$ is the norm ℓ_1 (ℓ_0 norm sparse regularizer can be approximated using the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 04:15:19 UTC from IEEE Xplore. Restrictions apply.

norm ℓ_1), according to Lemma 2, $g(\mathcal{Q}, \mathcal{U}_m)$ is semi-algebraic. Thus, $f(\mathcal{Q}, \mathcal{U}_m) + g(\mathcal{Q}, \mathcal{U}_m)$ is also semi-algebraic.

Therefore, according to Lemmas 1 and 2 and Condition 1, (x^t, z^t, y^t) is globally convergent to (x^*, z^*, y^*). Combine $x = (\mathcal{Q}, \mathcal{U}_m)$, $z = (\mathcal{J}, \mathcal{R}_m)$, and $y = (\mathcal{S}, \mathcal{B}_m)$, for each of the training example $\mathcal{T}_m$ $(m = 1, 2, \ldots, M)$, the constructed f and g satisfy Lemmas 1 and 2 and Condition 1, so the proposed algorithm is convergent, namely, $(\mathcal{Q}^t, \mathcal{U}_m^t, \mathcal{J}^t, \mathcal{R}_m^t, \mathcal{S}^t, \mathcal{B}_m^t)$ is globally convergent to $(\mathcal{Q}^*, \mathcal{U}_m^*, \mathcal{J}^*, \mathcal{R}_m^*, \mathcal{S}^*, \mathcal{B}_m^*)$. ■

Theorem 1 indicates that when appropriate initial variables are selected, the sequence generated by ADMM converges to its cluster point $(\mathcal{Q}^*, \mathcal{U}_m^*, \mathcal{J}^*, \mathcal{R}_m^*, \mathcal{S}^*, \mathcal{B}_m^*)$.

D. Deep Higher-Order SR Network for HSIs

Previously, we have built a higher-order tensor SR model, which can be used to obtain a general dictionary and corresponding SR. However, this model can only extract shallow features. Compared with shallow features, deep features have more complex, abstract, and robust representations.

Inspired by a CNN, we propose to construct a deep higher-order tensor SR network (DHTSRNet) to extract deep features from HSIs, as shown in Fig. 1. In DHTSRNet, it uses the same feature extraction module to extract HSI features, and each module uses the same PCA, convolution, and rectified linear units (ReLUs) operations. Fig. 2 shows the schematic of these three operations. Different from the traditional CNN, the proposed DHTSRNet contains only convolution and ReLU operation at each layer. Specifically, the feature $\boldsymbol{F}^l$ of any layer l can be obtained, which can be calculated by the following formula:

$$\begin{aligned} \boldsymbol{F}^l &= \boldsymbol{F}^{l-1} \oplus \mathcal{Q}^{l-1} \\ \boldsymbol{F}^l &= \max\left(0, \boldsymbol{F}^l - \boldsymbol{M}^l\right) \end{aligned} \tag{22}$$

where $\oplus$ denotes the 2-D convolution operator. $\boldsymbol{M}^l$ is the mean matrix of $\boldsymbol{F}^l$. In particular, when $l = 1$, $\boldsymbol{F}^0$ is the original input HSI data $\mathcal{T}_m$. In fact, in order to save memory and improve computational efficiency, we implement the higher-order tensor SR on image patches. That is, feature $\boldsymbol{F}^{l-1}$ of the previous layer is divided into nonoverlapping image patches and then constructed into a fourth-order tensor $\hat{\boldsymbol{F}}^{l-1} \in \mathbb{R}^{n_1 \times s \times n_2 \times n_3}$, which is then used as the input to the next layer. $\mathcal{Q}^{l-1} \in \mathbb{R}^{n_1 \times k \times n_2 \times n_3}$ represents the dictionary at the $l-1$th layer. In order to obtain deep $\mathcal{Q}^l$, we extend (5) into a deep model, then $\mathcal{Q}^l$ at the lth layer can be obtained by optimizing the following formula:

$$\min_{\mathcal{Q}^l, \mathcal{U}^l} \frac{1}{2} \sum_{m=1}^{s} \left\| \hat{\boldsymbol{F}}_m^l - \mathcal{Q}^l \otimes_{\mathrm{HO}} \mathcal{U}_m^l \right\|_F^2 + \lambda \left\| \mathcal{U}_m^l \right\|_{\underbrace{1, 1, \ldots, 1}_{d+1}} \tag{23}$$

where $\hat{\boldsymbol{F}}_m^l \in \mathbb{R}^{n_1 \times 1 \times n_2 \times n_3}$, $\hat{\boldsymbol{F}}^l = \{\hat{\boldsymbol{F}}_m^l\}_{m=1}^s$. $s = (wh/n_2 n_3)$ is the number of image patches at the lth layer, in which n_1 represents the first n_1 principal components retained by PCA, k is the number of filters in the dictionary $\mathcal{Q}^l$, and w and h represent the width and height of HSI, respectively. The optimization process of (23) is shown in Section III-B, and Theorem 1 proves that the optimization at the l-layer is globally convergent. In addition, Theorem 1 also shows that this deep model is convergent layer by layer.

In addition, we use local binary pattern (LBP) to extract spatial features from HSIs, which can be defined as

$$\mathcal{X} = \sum_{j=0}^{d^2-1} \mathcal{S}\left(P_j - P_c\right) 2^j \tag{24}$$

where

$$\begin{cases} \mathcal{S}\left(P_j - P_c\right) = 1, & \text{if } P_j > P_c \\ \mathcal{S}\left(P_j - P_c\right) = 0, & \text{else if } P_j \le P_c \end{cases} \tag{25}$$

where P_c represents local region centered at c, and its size is $d \times d$.

The detailed dimensional transformation at the l layer is as follows: for the input $\boldsymbol{F}^{l-1} \in \mathbb{R}^{w \times h \times n_1}$ (third-order tensor) at the $l-1$ layer, it is divided into nonoverlapping image patches, where w, h, and n_1 represent the width, height, and band number of the original HSI, respectively. Assuming that the size of each image patch is $n_2 \times n_3$, $s = (wh)/(n_2 n_3)$ image patches of size $n_2 \times n_3$ can be obtained. Then, it is extended into a fourth-order tensor $\hat{\boldsymbol{F}}^{l-1} \in \mathbb{R}^{n_1 \times s \times n_2 \times n_3}$, and trained by HTSR model ($\hat{\boldsymbol{F}}^{l-1}$ as the input data to be decomposed) to obtain dictionary $\mathcal{Q}^{l-1} \in \mathbb{R}^{n_1 \times k \times n_2 \times n_3}$, and convolved $\mathcal{Q}^{l-1}$ with the input $\boldsymbol{F}^{l-1}$ at the $l-1$ layer (2-D convolution operation along the two dimensions of $\mathcal{Q}^{l-1}$). It can use the MATLAB toolbox conv2 ($\boldsymbol{F}^{l-1}(:, :, i)$, $\mathcal{Q}^{l-1}(i, j, :, :)$, "same") ($i \in \{1, 2, \ldots, n_1\}$, $j \in \{1, 2, \ldots, k\}$) for operation, and the final result is a 3-D tensor. After dimensionality reduction by PCA, the output result is the same size as the input $\boldsymbol{F}^{l-1}$ at the $l-1$ layer.

Since convolution operation is used to extract deep features, the greater the number of convolutional kernels k, the greater the number of features extracted, and the size $n_2 \times n_3$ of each convolution kernel can extract the detailed information of features, that is, whether the feature information is sufficient or not affects the performance of the whole feature classification. Therefore, the number of convolutional kernels k and the spatial size $n_2 \times n_3$ of convolutional kernels are two very important parameters, and the setting of these two parameters will be discussed in detail in Section IV-B.

Algorithm 1 HSI Classification Framework

Input: The original HSI data $\mathcal{T}_m$; the dictionary $\mathcal{Q}^l$.
Output: Label $\boldsymbol{T}$.

```
for Layer i = 1 : l do
    Initialize t = 3;
    Applying PCA and local binary pattern to extract spectral, spatial feature, and combine H̃;
    Using (22) to obtain the feature F^l of each layer.
end for
Obtain LBP feature X;
Combining the the features of all layers F^1, F^2, ..., F^l, X and T_m to obtain F_fus,
Normalize the feature F_fus to obtain F̂_fus.
Classify the feature F̂_fus to obtain the class label T.
```

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 04:15:19 UTC from IEEE Xplore. Restrictions apply.

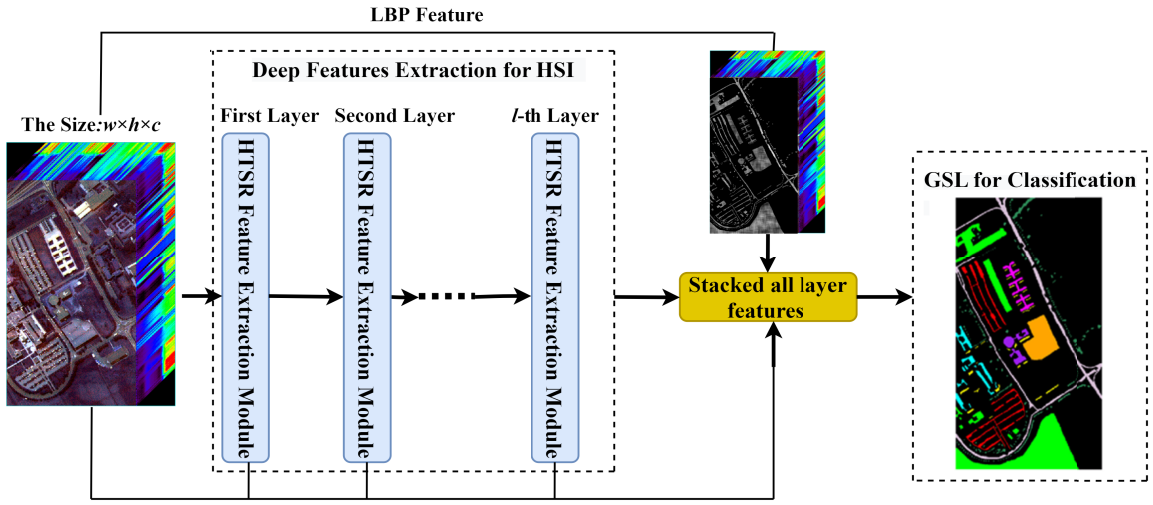


Fig. 1. DHTSRNet for HSI classification and the detailed architecture of the HTSR feature extraction module is shown in Fig. 2.

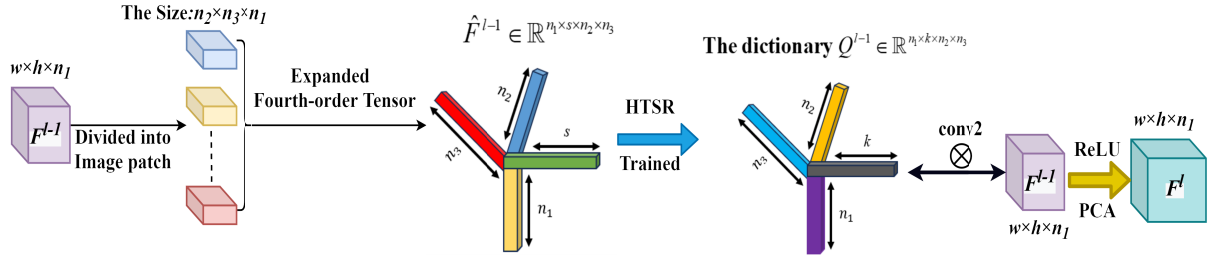


Fig. 2. HTSR feature extraction module, where "conv2" represents a 2-D convolution operation.

E. HSI Classification Based on DHTSRNet

Next, we will classify the deep features obtained using DHTSRNet in Section III-D. In order to improve classification accuracy, we combine the features of each layer. Specifically, combining the features of all layers $\boldsymbol{F}^1, \boldsymbol{F}^2, \ldots, \boldsymbol{F}^l$, LBP feature $\mathcal{X}$ and the original HSI data $\mathcal{T}_m$, we can get the combined feature $\mathcal{F}_{\text{fus}}$. For each dimension of $\mathcal{F}_{\text{fus}}$, it can be standardized

$$\hat{\mathcal{F}}_{\text{fus}}^{i} = \frac{\mathcal{F}_{\text{fus}}^{i} - \text{mean}\left(\mathcal{F}_{\text{fus}}^{i}\right)}{\text{var}\left(\mathcal{F}_{\text{fus}}^{i}\right)}, \quad i \in \{1, 2, \ldots, lp + 2m + t\} \tag{26}$$

where $\hat{\mathcal{F}}_{\text{fus}}^{j} \in \mathbb{R}^{rc\times 1}$ denotes the normalized data, mean$(\cdot)$ and var$(\cdot)$ represent the mean and variance, respectively. By concatenating all $\hat{\mathcal{F}}_{\text{fus}}^{j}$, the feature $\hat{\mathcal{F}}_{\text{fus}} \in \mathbb{R}^{rc\times(lp+2m+t)}$ is obtained, and we use GSL method to classify $\hat{\mathcal{F}}_{\text{fus}}$.

For GSL method [30], we need to divide $\hat{\mathcal{F}}_{\text{fus}}$ into training samples $\mathcal{Y}_{\text{lab}} = [\boldsymbol{y}_1, \boldsymbol{y}_2, \ldots, \boldsymbol{y}_l]$ and test samples $\mathcal{Y}_{\text{unlab}} = [\boldsymbol{y}_{l+1}, \boldsymbol{y}_{l+2}, \ldots, \boldsymbol{y}_N]$. For hyperspectral data with k-class, the GSL is used to classify, which can be solved by the following quadratic optimization problem:

$$\min_{\boldsymbol{T}} \text{tr}(\boldsymbol{T} - \boldsymbol{V})^T \boldsymbol{M}(\boldsymbol{T} - \boldsymbol{V}) + \text{tr}\left(\boldsymbol{T}^T \boldsymbol{G} \boldsymbol{T}\right) \tag{27}$$

where the trace function is represented by tr$(\cdot)$, $\boldsymbol{T} \in \mathbb{R}^{N\times k}$ represents the label to be predicted.

$$\boldsymbol{V} = \begin{pmatrix} v_{11} & \cdots & v_{1k} \\ \vdots & \ddots & \vdots \\ v_{N1} & \cdots & v_{Nk} \end{pmatrix}, \quad \begin{cases} v_{ij} = 1, & \text{if } \boldsymbol{y}_i \in j \\ v_{ij} = 0, & \text{otherwise.} \end{cases} \tag{28}$$

$\boldsymbol{M} \in \mathbb{R}^{N\times N}$, it is defined as follows:

$$\boldsymbol{M} = \begin{pmatrix} m_1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & m_N \end{pmatrix}, \quad \begin{cases} m_i = \delta_n, & \text{if } 1 \le i \le n \\ m_i = \delta_{N-n}, & \text{otherwise} \end{cases} \tag{29}$$

where δ_n and δ_{N-n} are two preset parameters. The graph Laplacian $\boldsymbol{G} \in \mathbb{R}^{N\times N}$ is calculated by

$$\boldsymbol{G} = \boldsymbol{L} - \boldsymbol{W} \tag{30}$$

where $\boldsymbol{W} = \{\omega_{ij}\}(i, j = 1, \ldots, N) \in \mathbb{R}^{N\times N}$, and ω_{ij} is defined as follows:

$$\omega_{ij} = e^{-\frac{\|x_i - x_j\|^2}{2\theta^2}} (i \neq j), \quad w_{ii} = 0 \tag{31}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 04:15:19 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DETAILED VALUES OF THE THREE DATASETS

	IP	PU	SA
Photographed location	North-western Indiana	Pavia, Northern Italy	Salinas Valley, California
Image size	145×145	610×340	512×217
Number of wavebands	200	103	204
Sensor	AVIRIS	ROSIS-03	AVIRIS
Number of classes	16	9	16
Total number of samples	10776	42776	54129

where ϑ is the preset parameter, $\boldsymbol{L}$ is defined by

$$\boldsymbol{L} = \begin{pmatrix} d_1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & d_N \end{pmatrix}, \quad d_i = \sum_{j=1}^{N} w_{ij}. \tag{32}$$

Algorithm 1 summarizes the whole framework of HSI classification.

TABLE II
EFFECT OF THE NUMBER OF NETWORK LAYERS (L) ON CLASSIFICATION PERFORMANCE ON THE THREE DATASETS

	1	3	5	10	12
IP	84.51	86.18	88.53	**89.64**	87.76
PU	85.16	87.14	87.79	**91.01**	84.63
SA	86.23	88.05	91.39	**95.47**	94.92

F. Complexity Analysis

The most time-consuming part of the algorithm includes the following three aspects: The first is using HTSR to solve the dictionary $\mathcal{Q}^l$ and SR $\mathcal{U}^l$; The second is the convolution operation, and the last is PCA. The complexity of the proposed DHTSRNet can be obtained by adding the total computational complexity of the above three aspects together with the computational complexity of LBP. Next, we analyze the computational complexity of each part to derive the overall computational complexity.

First, suppose the size of the dictionary $\mathcal{Q}^l$ at each layer l is $n_1 \times k \times n_2 \times n_3$, the dictionary learning subproblem involves solving $n_2 \times n_3$ linear systems of size $k \times k$, and the subproblem is solved entirely in the Fourier domain, which brings the computational complexity for solving the dictionary as $\mathcal{Q}^l$ to $\mathcal{O}(n_2 n_3 k^3) + \mathcal{O}(n_1 n_2 n_3 \log(n_2 n_3))$. As for the SR $\mathcal{U}^l$, the most expensive part is solving $\mathcal{U}^l$, which involves taking n_1 2-D Fourier transforms of size $n_2 \times n_3$ and solving $n_2 n_3$ linear systems of size $k \times k$, with a computational complexity of $\mathcal{O}(n_2 n_3 k^3) + \mathcal{O}(n_1 n_2 n_3 \log(n_2 n_3))$. Therefore, the computational complexity for solving the dictionary $\mathcal{Q}^l$ and SR $\mathcal{U}^l$ is $\mathcal{O}(2n_2 n_3 k^3) + \mathcal{O}(2n_1 n_2 n_3 \log(n_2 n_3))$. Second, each layer contains some convolution operations, and its total computational complexity is $\mathcal{O}(sn_1 whn_2 n_3 + swhn_1)$. Finally, The computational complexity of using PCA to reduce the dimensionality of the activation function is $\mathcal{O}(whk^2 + k^3 + whkn_1)$.

In addition, the computational complexity of LBP is $\mathcal{O}(whn_1 d^2 - whn_1)$.

Therefore, we add the computational complexity of solving the dictionary $\mathcal{Q}^l$ and SR $\mathcal{U}^l$, convolution operation, and PCA to obtain the computational complexity of each layer. Therefore, the computational complexity of each layer in the training phase is

$$\begin{aligned} &\mathcal{O}\left(2n_2 n_3 k^3\right) + \mathcal{O}(2n_1 n_2 n_3 \log(n_2 n_3)) + \mathcal{O}\left(whn_1 d^2 - whn_1\right) \\ &+ \mathcal{O}(sn_1 whn_2 n_3 + swhn_1) + \mathcal{O}\left(whk^2 + k^3 + whkn_1\right). \end{aligned} \tag{33}$$

If the total number of DHTSRNet layers is L, then L times the computational complexity of one layer obtained above is the total computational complexity of the entire deep network.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

A. Experimental Datasets

We will verify the effectiveness of the proposed method on three commonly used HSI datasets.

To comprehensively compare and analyze the proposed method, three mainstream HSI benchmark datasets are selected, including two types of field scenes: Indian Pines (IP)[1] and a type of urban scene: Pavia University (PU)[2]; and Salinas (SA). [3] Table I lists the detailed values of the three datasets, and their false color images and ground truth maps are provided in Section IV-C.

The evaluation metrics consist of overall accuracy (OA), average accuracy (AA), and Kappa coefficient (Kappa). To ensure the fairness and reliability of the experiments, all the results in this section are the mean of ten experimental results. Ten samples from each category were randomly sampled separately as the training set, and all the remaining samples were used as the test set. In Section IV-D, if the number of training samples changes, it will be reexplained. It should be noted here that to make it easier to set the initial parameters, we fixed the filter size and number to be the same on the dictionary of all layers.

B. Analysis and Setting of Parameters

In this section, we experimentally analyze the key parameters that can affect the feature extraction of HSIs. First, in deep learning methods, the number of layers often affects the performance of the deep learning method. More

[1]http://www.ehu.eus/ccwintco/index.php/Hyperspectral_Remote_Sensing_Scenes

[2]http://www.ehu.eus/ccwintco/index.php/Hyperspectral_Remote_Sensing_Scenes #Pavia_University_scene

[3]http://www.ehu.eus/ccwintco/index.php/Hyperspectral_Remote_Sensing_Scenes #Pavia_University_scene

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 04:15:19 UTC from IEEE Xplore. Restrictions apply.

TABLE III
EFFECT OF DIFFERENT SPATIAL SIZE FILTERS ($n_2 \times n_3$) IN DICTIONARY $\mathcal{Q}^l$ AT EACH LAYER l ON CLASSIFICATION PERFORMANCE ON THE THREE DATASETS

	Small size			Middle size			Large size		
	5×5	7×7	9×9	13×13	15×15	17×17	23×23	25×25	27×27
IP	79.75	81.26	83.19	85.96	86.23	87.16	86.55	88.23	**89.64**
PU	82.92	85.18	84.71	88.68	87.48	87.08	86.62	**91.01**	85.75
SA	83.61	84.34	86.45	88.75	92.11	88.58	**95.47**	95.21	94.93

network layers tend to improve the performance of the network. However, the more network layers, the more time deep learning methods spend in the training and testing phase. Therefore, selecting the appropriate number of network layers can not only obtain higher classification accuracy but also save time. In order to analyze the effect of the number of network layers on the feature extraction of HSIs, we studied the classification performance under different network layers $L \in \{1, 3, 5, 10, 12\}$. The results are shown in Table II, which show that when the number of network layers is equal to 1, the classification accuracy of the proposed method is low. The classification accuracy increases as the number of network layers increases, but when the number of network layers is greater than 10, the classification accuracy will decrease. Considering that the proposed method with $L = 10$ can obtain high-precision classification results, the number of network layers of the proposed method is set to 10 on the three datasets.

Second, the spatial size of the filter also affects the classification performance of the proposed method. Larger filters can extract better receptive fields, while smaller filters can obtain local details. To fully study the effect of filter size on classification performance, we selected both smaller filters, i.e., 5×5, 7×7, and 9×9, middle filters, 13×13, 15×15, and 17×17, and larger filters, 23×23, 25×25, and 27×27 for experiments. The experimental results are shown in Table III, which show that the classification accuracy of the proposed method with larger size filters on the three datasets is higher than those with smaller size filters and middle size filters. When the filter size of the proposed method is set to 27×27, 25×25, and 23×23 on three datasets, respectively, high classification accuracy is obtained.

Finally, the number of filters also affects the classification performance of the proposed method. More filters can obtain more discriminative features, but excessive filters lead to redundant features. To this end, we studied the influence of different filter numbers on the classification results, and the experimental results are shown in Table IV. It can be seen from the results in the table that when the number of filters is less than 50, the classification accuracy of the proposed method increases with the increase of the number of filters. When the number of filters is set to 50, 100, and 60, the proposed method achieves higher classification accuracy on the IP, PU, and SA datasets, respectively.

TABLE IV
EFFECT OF THE NUMBER OF FILTERS (k) IN THE DICTIONARY $\mathcal{Q}^l$ AT EACH LAYER l ON CLASSIFICATION PERFORMANCE ON THE THREE DATASETS

	10	20	30	40	50	60	100
IP	79.68	82.33	84.71	87.09	**89.64**	88.72	87.52
PU	80.14	83.26	85.68	86.83	88.05	90.11	**91.01**
SA	81.57	84.73	86.86	90.27	93.22	**95.47**	94.39

C. Classification Results

In order to evaluate the classification performance of DHTSRNet, we introduce some closely related hyperspectral classification methods for comparison, including deep tensor factorization (DTF) [39], GSL [50], a representative texture feature extraction method LBPs [51], CNN (LeNet) proposed by LeCun et al. [52], random patches network (RPNet) [53], spatial–spectral RPNet (SSRPNet) [30], deep higher-order tensor convolution sparse coding network (DHTCSCNet) [7], dual-branch deep SAFD network with GSL (DSAFD-Net_g) [54], and deep SAFD network (SAFDNet) [18]. In addition, some state-of-the-art deep learning methods such as the unsupervised 3-D convolutional auto-encoder feature extraction framework (3D-CAE) [55], the spectral–spatial transformer network (SSTN) [1] and the global contextual network (FullyContNet) embedded with a multiscale attention module [56] are also compared with the proposed method on the three datasets. Tables V–VII show the quantitative assessment. The classification maps of different methods are shown in Figs. 3–5. Here, the results of the comparison methods reported in our article are all derived from the results in their corresponding papers.

Among those closely related hyperspectral classification methods, SAFDNet has the best classification performance on IP dataset, but the OA of the proposed method is 5.1% higher than SAFDNet; RPNet has the best classification performance on PU dataset, but the OA of the proposed method is 10.5% higher than RPNet; LBP has the best classification performance on SA dataset, but the OA of the proposed method is 6% higher than LBP. For other state-of-the-art deep learning methods, such as 3D-CAE, SSTN and FullyContNet, SSTN has the best classification performance on IP and PU datasets, but the OAs of the proposed method are 4.2% higher and 0.8% than it, respectively, and FullyContNet has the best classification performance on the SA dataset, but the OA of the proposed method is about 0.6% higher than it.

In particular, although the proposed method uses the components LBP and GSL, the accuracy of the proposed method is significantly higher than that of the three datasets. This shows that a good FE helps improve classification performance. Although those closely related hyperspectral

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 04:15:19 UTC from IEEE Xplore. Restrictions apply.

TABLE V
CLASSIFICATION RESULTS (TEN TRAINING SAMPLES IN EACH CLASS) WITH DIFFERENT METHODS ON THE IP

Class	DTF	GSL	LBP	LeNet	RPNet	SSRPNet	DHTCSCNet	DSAFDNet_g	SAFDNet	3D-CAE	SSTN	FullyContNet	Ours
1	**100**	28.57	38.30	97.22	97.22	37.11	41.86	**100**	**100**	48.50	98.69	**100**	**100**
2	65.44	40.52	80.86	71.09	70.45	80.90	**92.21**	75.45	85.96	57.11	79.97	59.55	74.46
3	78.78	41.25	**90.72**	56.83	75.12	76.91	72.48	80.97	68.64	59.12	80.05	68.51	77.27
4	**95.15**	20.60	85.27	53.30	67.84	37.94	60.60	70.55	58.79	51.10	95.02	88.31	79.31
5	84.14	58.82	82.41	84.36	82.24	81.02	79.36	96.77	87.59	69.51	89.39	79.89	**99.77**
6	85.69	77.97	68.12	88.06	88.61	83.93	79.41	88.36	83.89	93.40	**97.56**	89.13	94.95
7	**100**	15.38	50.00	88.89	94.44	34.09	45.00	34.62	51.43	60.85	**100**	**100**	81.82
8	**100**	91.92	81.96	92.74	94.23	75.16	93.41	99.36	98.52	98.96	**99.81**	98.38	**100**
9	90.00	7.27	40.00	**100**	**100**	13.51	13.70	38.46	**100**	54.55	**100**	**100**	52.63
10	71.31	45.06	80.67	52.08	84.72	60.90	70.38	66.88	79.17	63.37	99.81	76.67	88.91
11	57.46	64.35	96.95	62.37	53.62	92.08	88.90	78.10	85.39	80.29	**100**	67.88	91.83
12	61.75	19.37	82.76	81.82	64.15	74.41	89.10	89.39	92.59	48.12	82.80	64.16	**94.09**
13	**100**	62.75	51.06	96.92	98.97	43.94	36.38	98.45	79.84	95.71	79.49	99.51	**100**
14	93.78	90.74	90.98	85.10	84.86	98.82	92.72	97.66	93.05	94.50	86.23	91.02	**99.76**
15	95.48	28.84	73.96	82.45	93.88	83.68	79.78	**100**	95.65	71.71	99.75	93.49	98.17
16	92.77	96.05	62.60	67.47	90.36	27.96	50.30	84.09	73.45	98.82	93.83	**99.03**	96.47
OA (%)	75.28	51.26	82.58	71.85	74.19	74.88	79.75	82.80	84.54	71.90	85.41	76.39	**89.64**
AA (%)	85.73	49.34	72.29	78.79	83.79	62.65	67.85	81.19	83.37	71.61	87.10	85.97	**89.34**
Kappa (%)	72.49	45.69	80.42	68.31	71.03	71.96	77.13	80.19	82.35	68.42	84.61	73.48	**88.16**
Training time (s)	53.69	–	–	**1.79**	4.25	27.67	25.63	63.75	72.59	1156	481.19	86.96	38.71
Test time (s)	1.35	42.71	24.82	1.17	1.72	1.86	13.95	14.37	37.76	5.22	13.08	**0.39**	10.73

TABLE VI
CLASSIFICATION RESULTS (TEN TRAINING SAMPLES IN EACH CLASS) WITH DIFFERENT METHODS ON THE PU

Class	DTF	GSL	LBP	LeNet	RPNet	SSRPNet	DHTCSCNet	DSAFDNet_g	SAFDNet	3D-CAE	SSTN	FullyContNet	Ours
1	50.42	95.93	72.53	69.13	84.96	70.87	79.01	95.95	91.58	**97.57**	92.66	83.29	89.67
2	50.48	84.95	88.14	73.83	74.44	89.18	77.90	76.64	84.04	93.97	88.89	83.20	**98.94**
3	80.95	41.84	62.97	80.33	74.15	37.95	**93.81**	88.53	48.47	70.92	83.43	73.31	87.46
4	25.64	61.43	34.99	91.72	92.99	58.54	51.18	**95.53**	59.38	92.88	90.35	95.08	88.74
5	85.47	97.35	66.65	**100**	73.98	70.12	90.27	99.85	98.60	98.11	99.58	98.67	99.18
6	70.05	27.35	68.02	73.98	79.76	42.75	48.16	**95.50**	52.47	55.70	91.84	89.63	93.86
7	90.83	40.09	87.59	96.59	88.79	27.40	84.37	87.92	21.53	72.21	**98.63**	89.63	88.77
8	44.34	56.91	32.93	82.63	85.38	48.77	69.85	47.35	61.97	91.00	**92.43**	85.38	89.66
9	34.90	**99.89**	35.32	83.46	86.34	41.68	50.00	98.05	99.78	**99.89**	97.83	98.37	99.36
OA (%)	53.96	62.24	63.48	77.21	80.45	58.89	71.45	76.24	68.51	85.18	90.20	85.54	**91.01**
AA (%)	59.23	67.30	61.01	83.52	82.31	54.14	71.62	87.26	68.65	85.81	92.83	88.51	**92.85**
Kappa (%)	43.50	53.71	54.59	71.44	75.08	49.85	61.46	66.69	58.39	80.94	88.04	81.44	**89.53**
Training time (s)	99.51	–	–	9.36	**3.96**	157.85	130.82	169.17	136.79	1168	904.81	149.73	129.36
Test time (s)	2.50	122.51	131.66	1.82	1.91	2.01	64.94	38.45	72.99	32.04	38.94	**0.54**	72.28

TABLE VII
CLASSIFICATION RESULTS (TEN TRAINING SAMPLES IN EACH CLASS) WITH DIFFERENT METHODS ON THE SA

Class	DTF	GSL	LBP	LeNet	RPNet	SSRPNet	DHTCSCNet	DSAFDNet_g	SAFDNet	3D-CAE	SSTN	FullyContNet	Ours
1	64.03	**99.90**	95.13	95.75	98.40	96.17	71.48	78.48	57.17	96.17	94.58	97.72	96.39
2	87.30	98.91	97.29	81.46	97.77	98.69	68.93	**99.94**	97.43	98.88	98.50	95.68	99.89
3	93.03	90.25	97.34	99.90	99.03	49.34	50.79	91.96	84.62	92.26	93.75	**99.96**	99.75
4	72.33	97.23	82.51	**99.86**	99.71	96.44	56.57	94.15	82.48	97.87	98.91	98.79	97.31
5	69.27	98.19	92.09	98.05	97.26	94.87	75.30	**100**	97.62	96.70	95.43	97.15	96.97
6	78.88	99.95	96.89	99.42	98.23	88.25	96.66	99.57	99.94	**99.96**	99.46	99.54	97.05
7	94.62	96.81	87.30	95.80	91.01	84.74	83.23	99.69	95.13	97.95	99.20	99.38	**100**
8	71.74	56.83	84.10	76.14	55.92	82.31	61.24	**98.55**	98.44	82.72	76.15	87.07	97.36
9	98.72	97.57	96.77	98.77	99.02	81.09	86.74	94.10	62.82	99.27	99.15	99.82	**99.85**
10	64.93	92.31	92.70	78.00	92.75	97.25	83.65	93.16	93.63	87.27	91.77	95.74	**97.74**
11	76.09	86.13	95.16	99.34	**99.62**	69.05	98.75	98.09	95.83	78.69	97.42	99.27	97.87
12	64.89	93.05	97.99	99.48	97.81	84.52	51.86	93.07	93.62	94.83	99.14	**99.72**	97.68
13	85.87	88.01	97.74	95.58	94.26	58.83	32.33	99.65	76.82	94.17	**99.82**	99.49	96.86
14	58.02	93.58	82.78	97.36	96.89	66.11	41.53	98.19	63.60	95.02	**99.44**	98.82	90.83
15	**93.48**	39.46	83.97	58.93	60.39	50.96	46.68	55.99	49.72	63.48	81.03	89.83	93.69
16	96.49	97.65	71.59	98.27	98.78	69.04	62.38	**100**	43.58	91.13	96.03	97.15	99.50
OA (%)	81.47	79.47	89.47	85.95	83.52	75.10	67.55	86.39	70.32	87.86	90.78	94.88	**95.47**
AA (%)	79.36	89.11	90.71	92.01	92.30	79.23	66.76	93.41	80.78	91.65	94.99	97.20	**97.42**
Kappa (%)	79.56	77.16	88.28	84.37	81.72	72.43	63.89	84.93	67.43	86.54	89.77	94.31	**94.97**
Training time (s)	114.57	–	–	6.53	**6.03**	329.56	256.67	232.51	304.60	1159	1376.63	160.59	147.29
Test time (s)	2.89	239.12	298.42	4.66	4.98	5.11	82.06	53.84	158.65	26.4	32.82	**0.62**	106.18

classification methods are close to our method in terms of components, the classification accuracy of the proposed method is significantly higher than that of them in terms of classification accuracy. The essential difference between these methods lies in the different ways of feature extraction, leading to different classification results. The high accuracy achieved by the proposed method once again verifies the effectiveness of the proposed method in feature extraction. In addition, for the same type of method DTF, the classification accuracy of the proposed method on the three datasets is 14.3%, 37%, and 14% higher than the corresponding accuracy of DTF, respectively. In addition, compared with other methods, the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 04:15:19 UTC from IEEE Xplore. Restrictions apply.

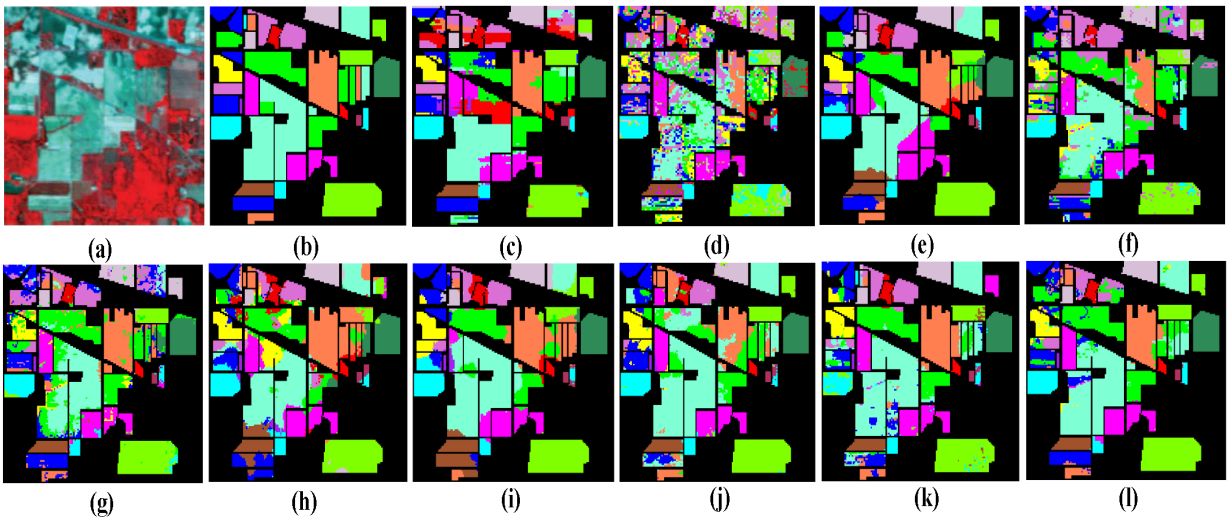


Fig. 3. Visualization of classification results of different methods on the IP dataset. (a) False color maps. (b) Real feature marker. (c) DTF (75.28%). (d) GSL (51.26%). (e) LBP (82.58%). (f) LeNet (71.85%). (g) RPNet (74.19%). (h) SSRPNet (74.88%). (i) DHTCSCNet (79.75%). (j) DSAFDNet_g (82.80%). (k) SAFDNet (84.54%). (l) Ours (89.64%).

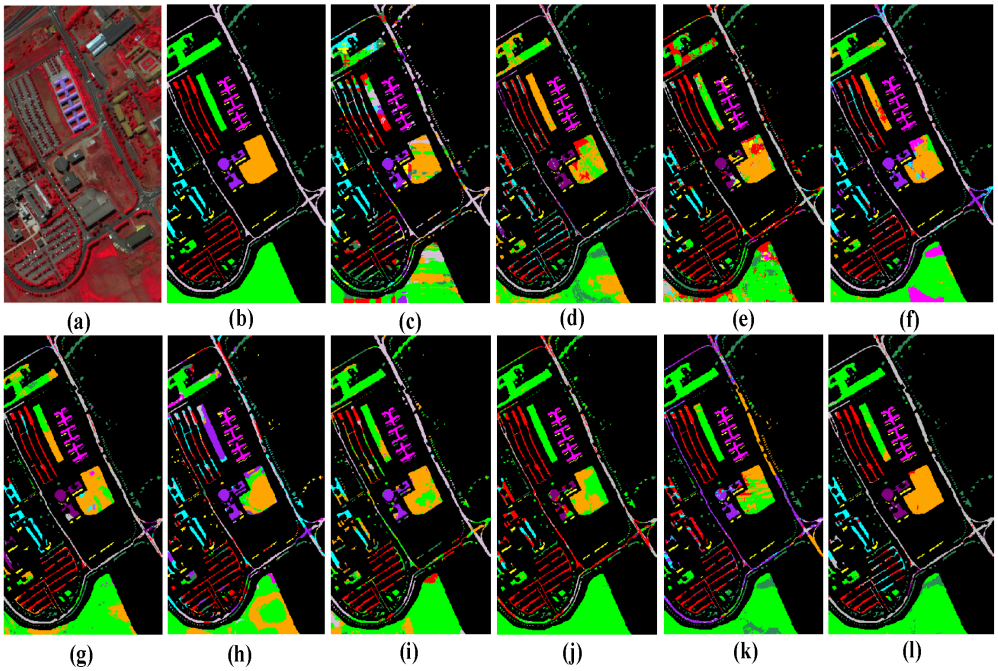


Fig. 4. Visualization of classification results of different methods on the PU dataset. (a) False color maps. (b) Real feature marker. (c) DTF (53.96%). (d) GSL (62.24%). (e) LBP (63.48%). (f) LeNet (77.21%). (g) RPNet (80.45%). (h) SSRPNet (58.89%). (i) DHTCSCNet (71.45%). (j) DSAFDNet_g (76.24%). (k) SAFDNet (68.51%). (l) Ours (91.01%).

proposed method obtains higher AA and Kappa. This also illustrates the effectiveness of the proposed deep HTSR model.

For further comparison, the full-domain classification maps of different methods on the three types of scenes are produced, as shown in Figs. 3–5, where Table VIII shows the correspondence between colors and categories on three datasets. The methods with low classification accuracy, such as GSL, generate a large amount of scatter noise. However, LBP exhibits a slightly higher classification precision, but its extracted texture features result in a distorted classification map, whereas the excessive texturization renders the classification map less visually appealing. This also indicates that methods with high classification accuracy do not necessarily produce a full-domain classification map that meets the application requirements. Although the classification maps of RPNet, SSRPNet, DHTCSCNet, SAFDNet, and DSAFDNet_g exhibit a significant noise reduction, the overall visual effect is still poor. By contrast, the proposed method is shown to obtain classification map results that are the closest to the original image and ground truth maps.

D. Analysis and Discussion

In order to further investigate the effectiveness of the proposed method, we did the following experiments.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 04:15:19 UTC from IEEE Xplore. Restrictions apply.

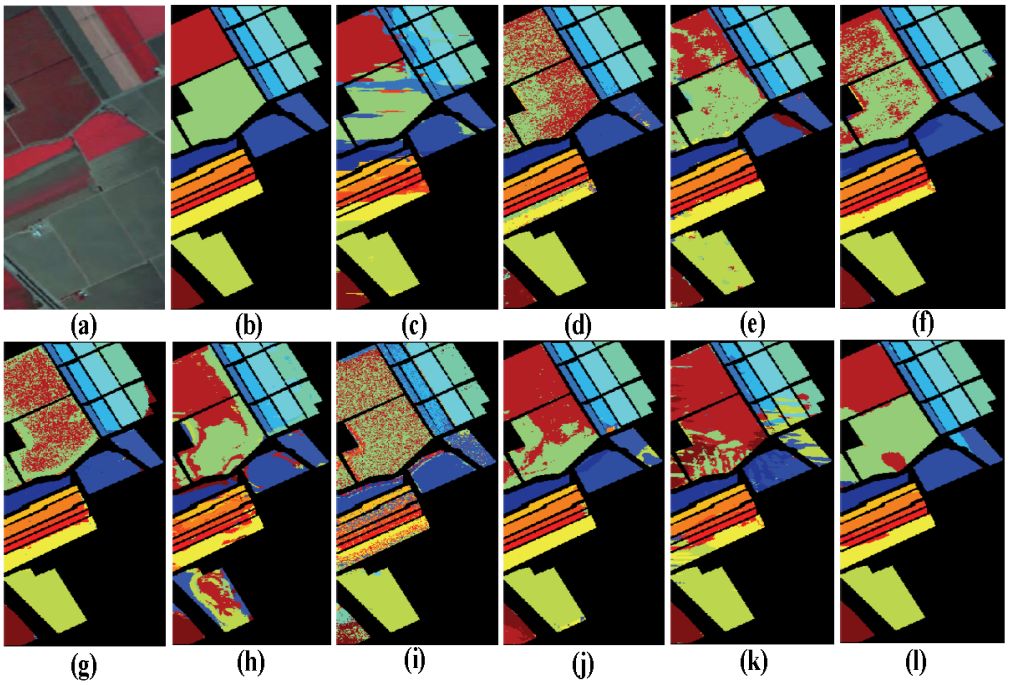


Fig. 5. Visualization of classification results of different methods on the SA dataset. (a) False color maps. (b) Real feature marker. (c) DTF (81.47%). (d) GSL (79.47%). (e) LBP (89.47%). (f) LeNet (85.95%). (g) RPNet (83.52%). (h) SSRPNet (75.10%). (i) DHTCSCNet (67.55%). (j) DSAFDNet_g (86.39%). (k) SAFDNet (70.32%). (l) Ours (95.47%).

1) Impact of Various Training Sample Sizes: For a more comprehensive comparison of adaptability of different methods for various training sample sizes, the training sample sizes are increased to 10, 20, 30, 50, 80, 100, and 150 step-by-step and the experimental results are shown in Fig. 6. As we can observe, the proposed method shows the highest classification performance for various training sample sizes on three datasets. Even when samples are sufficient, the classification accuracy of the proposed method is close to 100%. The good performance in various scenes fully verifies that the deep features extracted by the proposed method have a strong discriminative ability.

2) Convergence of the Proposed Method: The proposed method uses a large number of network layers (ten-layer networks). In order to facilitate the analysis of the convergence of the network, we only analyze the convergence of the first three layers of the network, and the results on the three datasets are shown in Fig. 7. It can be seen from the figure that the proposed method can converge quickly with fewer iterations on three datasets. This is consistent with the proof of convergence of Theorem 1. It should be noted here that even when the number of network layers is greater than 3, it is still convergent on the three datasets.

3) Comparison With Transformer-Based Classification Methods: In order to illustrate the advantages of the proposed method, we also compare it with transformer-based classification methods including HIT [57], GAHT [58], SSFTT [59], and DBMST [60]. We also chose to compare the CNN-type methods that are closest to the proposed methods, including CNN (LeNet) proposed by LeCun et al. [52] and RPNet [53]. In addition, in order to verify the performance of the proposed method on more challenging datasets, we conducted a comparative experiment on Houston 2013 Dataset [61]. Table IX gives the classification results of OA, AA, and Kappa on IP, PU, SA, and Houston 2013 datasets, respectively. It can be seen from the results in the table that the transformer-based classification method is better than that of CNN-based classification methods. However, the proposed method has the highest OAs on the four datasets. Among the transformer-based methods, DBMST achieved better classification results on all four datasets. In contrast to DBMST, the OAs and Kappa of the proposed method are the highest on the four datasets, while the AA of the proposed method is the highest except for the SA dataset. This shows that even compared with the SOTA classification algorithms, the proposed method has great advantages. In particular, the proposed method still achieves better classification results on more challenging datasets (Houston 2013). This also verifies the effectiveness of the proposed method. It should be noted here that the number of training samples for all comparison methods was consistent with [60].

4) Ablation Study for LBP: LBP can be used to extract the spatial information of HSIs, which is helpful to improve classification performance. To verify the effectiveness of LBP, we performed ablation experiments on four datasets: IP, PU, SA, and Houston 2013. The results are shown in Table X. From the table, it can be seen that on the IP dataset, the OA with LBP is 12.70% higher than that without LBP. For the PU dataset, compared to the absence of LBP, the OA with LBP improved by 23.52%. For the SA dataset, without LBP, the OA would decrease by 5.51%. For the Houston 2013 dataset, compared to the absence of LBP, the OA with LBP improved

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 04:15:19 UTC from IEEE Xplore. Restrictions apply.

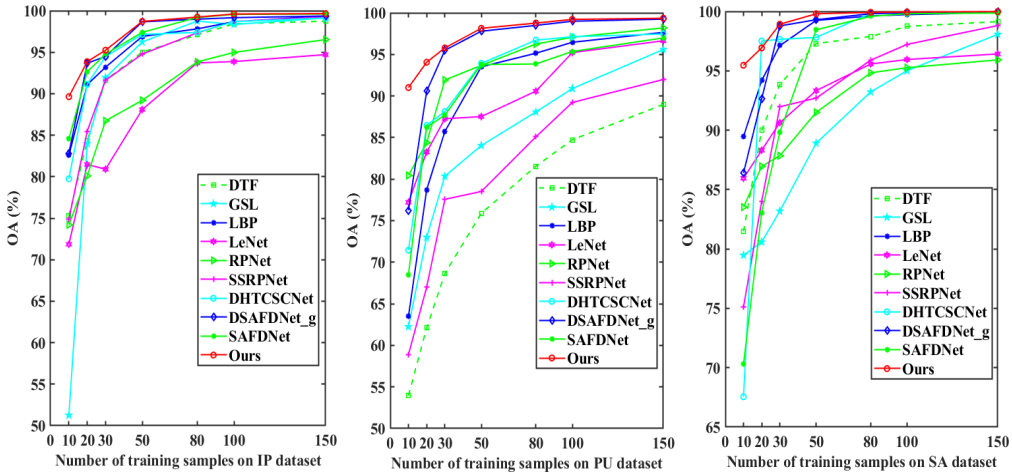


Fig. 6. Classification performance of different methods according to the variation of number of training samples on three datasets.

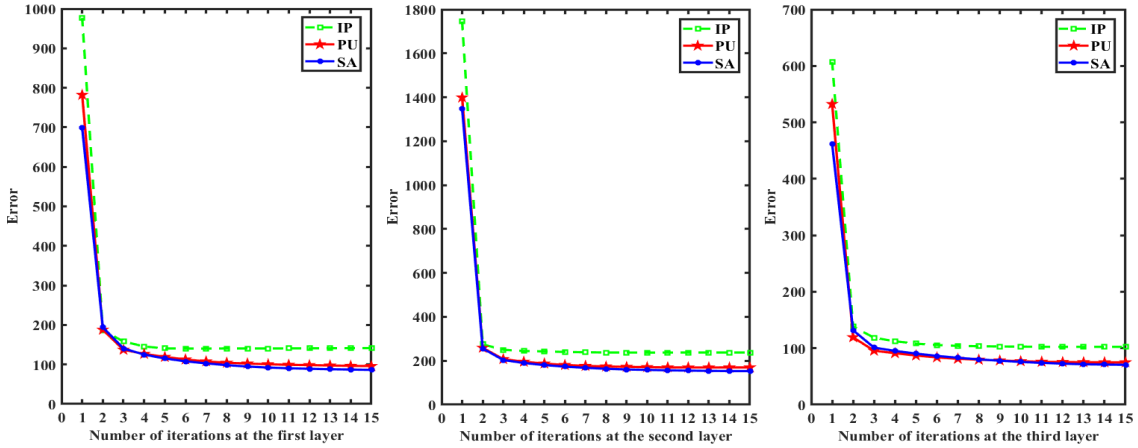


Fig. 7. Convergence of the first three layers of the proposed method on three datasets.

TABLE VIII
CORRESPONDENCE BETWEEN COLORS AND CATEGORIES ON THREE DATASETS

IP			Pu			SA		
Class	Class name	Color block	Class name	Color block		Class name	Color block	
1	Corn Notill		Asphalt			Brocoli green weeds 1		
2	Corn Mintill		Meadows			Brocoli green weeds 2		
3	Corn		Gravel			Fallow		
4	Grass Pasture		Trees			Fallow rough plow		
5	Grass Trees		Painted metal sheets			Fallow smooth		
6	Hay Windrowed		Bare Soil			Stubble		
7	Soybean Notill		Bitumen			Celery		
8	Soybean Mintill		Self-Blocking Bricks			Grapes untrained		
9	Soybean Clean		Shadows			Soil vinyard develop		
10	Wheat					Corn senesced green weeds		
11	Woods					Lettuce romaine 4wk		
12	Buildings Grass Trees Drives					Lettuce romaine 5wk		
13	Stone Steel Towers					Lettuce romaine 6wk		
14	Alfalfa					Lettuce romaine 7wk		
15	Grass Pasture Mowed					Vinyard untrained		
16	Oats					Vinyard vertical trellis		

by 2.93%. In summary, the ablation experiments of the LBP on four datasets have demonstrated the effectiveness of LBP.

5) Running Time: Tables V–VII show test time and training time of different methods. The deep-learning-based methods involve both training and testing steps, from the results in the table, the test time of FullyContNet is the lowest. This is mainly because the results of the two methods SSTN and FullyContNet are executed on the GPU, while the other methods are executed on the CPU, and obviously the GPU runs faster than the CPU. However, the training time of the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 04:15:19 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
COMPARISON WITH CNNS-BASED TYPE AND TRANSFORMER-BASED CLASSIFICATION METHODS ON FOUR DATASETS

		LeNet	RPNet	HIT	GAHT	SSFTT	DBMST	Ours
IP	OA	94.66	96.04	90.46	83.82	98.07	98.84	**99.06**
	AA	83.81	92.94	91.96	87.02	97.13	98.24	**98.64**
	Kappa	93.90	95.47	89.09	81.46	97.79	98.68	**98.93**
PU	OA	92.34	92.33	86.62	86.68	96.57	97.69	**98.19**
	AA	86.86	86.07	82.22	84.60	95.56	96.82	**98.09**
	Kappa	89.80	89.73	82.07	82.09	95.44	96.94	**97.59**
SA	OA	94.25	92.31	93.51	91.50	98.25	98.97	**99.24**
	AA	96.99	95.44	95.01	94.46	98.66	**99.18**	99.03
	Kappa	93.59	91.44	92.78	90.55	98.05	98.72	**99.18**
Houston	OA	93.79	94.29	89.96	89.92	97.43	97.72	**97.91**
	AA	92.07	93.40	90.05	90.62	97.58	97.77	**97.81**
	Kappa	93.28	93.83	89.14	89.10	97.22	97.53	**97.74**

TABLE X
IMPACT OF LBP ON OA OF THE PROPOSED METHOD (TEN TRAINING SAMPLES IN EACH CLASS) ON DIFFERENT DATASETS (%)

With or Without LBP	IP	PU	SA	Houston
With LBP	**89.64**	**91.01**	**95.47**	**87.41**
Without LBP	76.90	67.49	89.96	84.48

proposed method is less than that of FullyContNet. Since the proposed method is directly used for feature extraction on high-order tensor data, it takes more time to compute than low-order tensor data. Among these methods executed on the CPU, RPNet takes the shortest time, mainly because it directly uses low-dimensional image patches as the initial weights of deep networks to quickly extract network features, and the depth of the network is only five layers. LeNet uses a low-dimensional random initialization matrix as the initial weight of the deep network, and the number of layers in the network is also 5, which can be faster in feature extraction. In contrast, the proposed method uses a ten-layer network and is tested on higher-order tensor-type data, which requires more time for feature extraction.

In summary, the proposed method can directly process higher-order tensor data and extract deep structure information, which is helpful in improving classification performance. However, when the proposed method is tested on higher-order tensor-type data, it takes a long time to train the parameters in the optimization stage. In the future, we will consider optimizing on GPU and choose a faster iterative algorithm to solve the proposed HTSR model.

V. CONCLUSION

In this article, we propose a high-order tensor SR model, that can handle arbitrary-order tensor-type data and train deep high-order filters. Furthermore, the HTSR model is extended to the deep HTSR model, which can be used to extract deep features. Based on a deep HTSR model, an HSI classification method based on GSL is constructed, which can obtain better classification results in the case of small training samples.

In the future, we will continue to investigate methods for quickly computing higher-order tensor-type data and design a more efficient FE.

REFERENCES

[1] Z. Zhong, Y. Li, L. Ma, J. Li, and W.-S. Zheng, "Spectral–spatial transformer network for hyperspectral image classification: A factorized architecture search framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5514715.

[2] B. Liu, Y. Sun, A. Yu, Z. Xue, and X. Zuo, "Hyperspectral meets optical flow: Spectral flow extraction for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 32, pp. 5181–5196, 2023.

[3] B. Zhang, Y. Chen, Z. Li, S. Xiong, and X. Lu, "SANet: A self-attention network for agricultural hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5501315.

[4] C. Sammut and G. I. Webb, "Principal component analysis," in *Encyclopedia of Machine Learning*. Boston, MA, USA: Springer, 2010, doi: 10.1007/978-0-387-30164-8_665.

[5] J. Xia, N. Falco, J. A. Benediktsson, P. Du, and J. Chanussot, "Hyperspectral image classification with rotation random forest via KPCA," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 10, no. 4, pp. 1601–1609, Apr. 2017.

[6] M. Wu et al., "Heterogeneous cuckoo search-based unsupervised band selection for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5500616.

[7] C. Cheng, H. Li, J. Peng, W. Cui, and L. Zhang, "Deep high-order tensor convolutional sparse coding for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5520611.

[8] S. Jia, J. Hu, J. Zhu, X. Jia, and Q. Li, "Three-dimensional local binary patterns for hyperspectral imagery classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 4, pp. 2399–2413, Apr. 2017.

[9] F. Gao, Q. Wang, J. Dong, and Q. Xu, "Spectral and spatial classification of hyperspectral images based on random multi-graphs," *Remote. Sens.*, vol. 10, no. 8, pp. 1271–1290, Jul. 2018.

[10] J. Liu, Z. Wu, Z. Wei, L. Xiao, and L. Sun, "Spatial–spectral kernel sparse representation for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 6, no. 6, pp. 2462–2471, Dec. 2013.

[11] A. V. Miclea, R. Terebes, and S. Meza, "One dimensional convolutional neural networks and local binary patterns for hyperspectral image classification," in *Proc. IEEE Int. Conf. Autom., Quality Test., Robot. (AQTR)*, May 2020, pp. 1–6.

[12] H. Lee and H. Kwon, "Going deeper with contextual CNN for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 26, no. 10, pp. 4843–4855, Oct. 2017.

[13] Q. Zhang, J. Sun, G. Zhong, and J. Dong, "Random multi-graphs: A semi-supervised learning framework for classification of high dimensional data," *Image Vis. Comput.*, vol. 60, pp. 30–37, Apr. 2017.

[14] W. Wang, S. Dou, Z. Jiang, and L. Sun, "A fast dense spectral–spatial convolution network framework for hyperspectral images classification," *Remote Sens.*, vol. 10, no. 7, p. 1068, Jul. 2018.

[15] A. B. Hamida, A. Benoit, P. Lambert, and C. B. Amar, "3-D deep learning approach for remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 8, pp. 4420–4434, Aug. 2018.

[16] W. Song, S. Li, L. Fang, and T. Lu, "Hyperspectral image classification with deep feature fusion network," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 6, pp. 3173–3184, Jun. 2018.

[17] X. Li, M. Ding, and A. Pižurica, "Deep feature fusion via two-stream convolutional neural network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 4, pp. 2615–2629, Apr. 2020.

[18] C. Cheng, L. Zhang, H. Li, L. Dai, and W. Cui, "A deep stochastic adaptive Fourier decomposition network for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 33, pp. 1080–1094, 2024.

[19] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D–2-D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Feb. 2020.

[20] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[21] J. He, L. Zhao, H. Yang, M. Zhang, and W. Li, "HSI-BERT: Hyperspectral image classification using the bidirectional encoder representation from transformers," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 1, pp. 165–178, Jan. 2020.

[22] C. Pu, H. Huang, and L. Yang, "An attention-driven convolutional neural network-based multi-level spectral–spatial feature learning for hyperspectral image classification," *Expert Syst. Appl.*, vol. 185, Dec. 2021, Art. no. 115663.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 04:15:19 UTC from IEEE Xplore. Restrictions apply.

[23] H. Liu, W. Li, X.-G. Xia, M. Zhang, C.-Z. Gao, and R. Tao, "Central attention network for hyperspectral imagery classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 11, pp. 8989–9003, Nov. 2023.
[24] H. Liu, W. Li, X.-G. Xia, M. Zhang, and R. Tao, "Multiarea target attention for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5524916.
[25] Z. Ye, J. Wang, H. Liu, Y. Zhang, and W. Li, "Adaptive domain-adversarial few-shot learning for cross-domain hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5532017.
[26] M. Zhang, W. Li, Y. Zhang, R. Tao, and Q. Du, "Hyperspectral and LiDAR data classification based on structural optimization transmission," *IEEE Trans. Cybern.*, vol. 53, no. 5, pp. 3153–3164, May 2023.
[27] M. Zhang, W. Li, X. Zhao, H. Liu, R. Tao, and Q. Du, "Morphological transformation and spatial–logical aggregation for tree species classification using hyperspectral imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, pp. 1–12, 2023, Art. no. 5501212.
[28] M. Zhang, X. Zhao, W. Li, Y. Zhang, R. Tao, and Q. Du, "Cross-scene joint classification of multisource data with multilevel domain adaption network," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Apr. 6, 2023, doi: 10.1109/TNNLS.2023.3262599.
[29] Z. Wang, S. Zhao, G. Zhao, and X. Song, "Dual-branch domain adaptation few-shot learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, Jan. 2024, Art. no. 5506116, doi: 10.1109/TGRS.2024.3356199.
[30] C. Cheng, H. Li, J. Peng, W. Cui, and L. Zhang, "Hyperspectral image classification via spectral–spatial random patches network," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 4753–4764, 2021.
[31] J. Chen, S. Yang, Z. Wang, and H. Mao, "Efficient sparse representation for learning with high-dimensional data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 8, pp. 4208–4222, Aug. 2023.
[32] C.-N. Li, Y.-H. Shao, W. Yin, and M.-Z. Liu, "Robust and sparse linear discriminant analysis via an alternating direction method of multipliers," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 3, pp. 915–926, Mar. 2020.
[33] J. Peng, L. Li, and Y. Y. Tang, "Maximum likelihood estimation-based joint sparse representation for the classification of hyperspectral remote sensing images," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 6, pp. 1790–1802, Jun. 2019.
[34] L. Wang, R. Chan, and T. Zeng, "Probabilistic semi-supervised learning via sparse graph structure learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 2, pp. 853–867, Feb. 2020.
[35] C. Huang, C. C. Loy, and X. Tang, "Discriminative sparse neighbor approximation for imbalanced learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 5, pp. 1503–1513, May 2018.
[36] J. Chen, H. Zhang, H. Mao, Y. Sang, and Z. Yi, "Symmetric low-rank representation for subspace clustering," *Neurocomputing*, vol. 173, pp. 1192–1202, Jan. 2016.
[37] Y. Gao, J. Ma, and A. L. Yuille, "Semi-supervised sparse representation based classification for face recognition with insufficient labeled samples," *IEEE Trans. Image Process.*, vol. 26, no. 5, pp. 2545–2560, May 2017.
[38] C. Lu, J. Feng, Y. Chen, W. Liu, Z. Lin, and S. Yan, "Tensor robust principal component analysis with a new tensor nuclear norm," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 925–938, Apr. 2020.
[39] J. Chen, W. Zhang, Y. Qian, and M. Ye, "Deep tensor factorization for hyperspectral image classification," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2018, pp. 4788–4791.
[40] W. Sun, S. Chen, L. Huang, H. C. So, and M. Xie, "Deep convolutional neural network compression via coupled tensor decomposition," *IEEE J. Sel. Topics Signal Process.*, vol. 15, no. 3, pp. 603–616, Apr. 2021.
[41] Y. Xu, Z. Wu, J. Chanussot, and Z. Wei, "Hyperspectral images super-resolution via learning high-order coupled tensor ring representation," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 11, pp. 4747–4760, Nov. 2020.
[42] J. Xue, Y. Zhao, S. Huang, W. Liao, J. C. Chan, and S. G. Kong, "Multilayer sparsity-based tensor decomposition for low-rank tensor completion," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 11, pp. 6916–6930, Nov. 2022.
[43] D. Jin, J. Liu, J. Yang, and Z. Wu, "High-order coupled fully connected tensor network decomposition for hyperspectral image super-resolution," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.
[44] C. I. Kanatsoulis, X. Fu, N. D. Sidiropoulos, and W.-K. Ma, "Hyperspectral super-resolution: A coupled tensor factorization approach," *IEEE Trans. Signal Process.*, vol. 66, no. 24, pp. 6503–6517, Dec. 2018.
[45] S. Li, R. Dian, L. Fang, and J. M. Bioucas-Dias, "Fusing hyperspectral and multispectral images via coupled sparse tensor factorization," *IEEE Trans. Image Process.*, vol. 27, no. 8, pp. 4118–4130, Aug. 2018.
[46] R. Dian, S. Li, and L. Fang, "Learning a low tensor-train rank representation for hyperspectral image super-resolution," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 9, pp. 2672–2683, Sep. 2019.
[47] Y. Chen, J. Zeng, W. He, X.-L. Zhao, and T.-Z. Huang, "Hyperspectral and multispectral image fusion using factor smoothed tensor ring decomposition," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–17, 2022, Art. no. 5515417.
[48] A. Bibi and B. Ghanem, "High order tensor formulation for convolutional sparse coding," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 1790–1798.
[49] G.-J. Peng, "Adaptive ADMM for dictionary learning in convolutional sparse representation," *IEEE Trans. Image Process.*, vol. 28, no. 7, pp. 3408–3422, Jul. 2019.
[50] L. Ma, A. Ma, C. Ju, and X. Li, "Graph-based semi-supervised learning for spectral–spatial hyperspectral image classification," *Pattern Recognit. Lett.*, vol. 83, pp. 133–142, Nov. 2016.
[51] W. Li, C. Chen, H. Su, and Q. Du, "Local binary patterns and extreme learning machine for hyperspectral imagery classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 7, pp. 3681–3693, Jul. 2015.
[52] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
[53] Y. Xu, B. Du, F. Zhang, and L. Zhang, "Hyperspectral image classification via a random patches network," *ISPRS J. Photogramm. Remote Sens.*, vol. 142, pp. 344–357, Aug. 2018.
[54] C. Cheng, L. Zhang, H. Li, and W. Cui, "A dual-branch deep stochastic adaptive Fourier decomposition network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5521515.
[55] S. Mei, J. Ji, Y. Geng, Z. Zhang, X. Li, and Q. Du, "Unsupervised spatial–spectral feature learning by 3D convolutional autoencoder for hyperspectral classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 6808–6820, Sep. 2019.
[56] D. Wang, B. Du, and L. Zhang, "Fully contextual network for hyperspectral scene parsing," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5501316.
[57] X. Yang, W. Cao, Y. Lu, and Y. Zhou, "Hyperspectral image transformer classification networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5528715.
[58] S. Mei, C. Song, M. Ma, and F. Xu, "Hyperspectral image classification using group-aware hierarchical transformer," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5539014.
[59] L. Sun, G. Zhao, Y. Zheng, and Z. Wu, "Spectral–spatial feature tokenization transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5522214.
[60] C. Shi, S. Yue, and L. Wang, "A dual-branch multiscale transformer network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5504520.
[61] C. Debes et al., "Hyperspectral and LiDAR data fusion: Outcome of the 2013 GRSS data fusion contest," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2405–2418, Jun. 2014.

Chunbo Cheng received the B.Sc. degree in mathematics and applied mathematics from Yangtze University, Jingzhou, China, in 2015, and the Ph.D. degree from the School of Mathematics and Statistics, Huazhong University of Science and Technology, Wuhan, China, in 2020.

He is currently an Associate Professor with the College of Mathematical and Physical Sciences, Hubei Polytechnic University, Huangshi, China. His research interests include pattern recognition, machine learning, and computer vision.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 04:15:19 UTC from IEEE Xplore. Restrictions apply.

Liming Zhang (Senior Member, IEEE) received the M.S. degree in signal processing from the Nanjing University of Science and Technology, Nanjing, China, in 1990, and the Ph.D. degree in image processing from the University of New England, Armidale, NSW, Australia, in 2001.

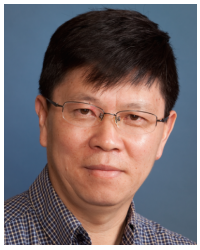
She is currently an Assistant Professor with the Faculty of Science and Technology, University of Macau, Macau, China. Her research interests include signal processing, image processing, and computer vision.

Hong Li (Member, IEEE) received the M.Sc. degree in mathematics and the Ph.D. degree in pattern recognition and intelligence control from the Huazhong University of Science and Technology, Wuhan, China, in 1986 and 1999, respectively.

She is currently a Professor with the School of Mathematics and Statistics, Huazhong University of Science and Technology. Her research interests include approximation theory, wavelet analysis, learning theory, neural networks, signal processing, and pattern recognition.

Wenjing Cui received the B.S. degree from Henan Normal University, Xinxiang, Henan, China, in 2015, and the M.S. degree from the Wuhan University of Technology, Wuhan, China, in 2018.

She is currently an Assistant with the College of Mathematical and Physical Sciences, Hubei Polytechnic University, Huangshi, China. Her research interests include machine learning and econometric analysis.



Junbin Gao (Member, IEEE) received the B.Sc. degree in computational mathematics from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 1982, and the Ph.D. degree from the Dalian University of Technology, Dalian, China, in 1991.

From 1982 to 2001, he was an Associate Lecturer, a Lecturer, an Associate Professor, and a Professor with the Department of Mathematics, HUST. From 2001 to 2005, he was a Senior Lecturer and a Lecturer in computer science at the University of New England, Armidale, NSW, Australia. He was a Professor of computer science at the School of Computing and Mathematics, Charles Sturt University, Bathurst, NSW, Australia. He is currently a Professor of big data analytics with the University of Sydney Business School, The University of Sydney, Camperdown, NSW, Australia. His main research interests include machine learning, data analytics, Bayesian learning and inference, and image analysis.

Yuxiao Cun received the B.Sc. degree in mathematics and applied mathematics from the Chongqing University of Arts and Sciences, Chongqing, China, in 2014, and the M.S. degree from the School of Mathematics and Statistics, Yunnan University, Kunming, China, in 2017.

She is currently a Lecturer with the Public Basic Course Teaching Department, West Yunnan University of Applied Sciences, Lincang, China. Her research interests include comprehensive evaluation and applied statistics.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 04:15:19 UTC from IEEE Xplore. Restrictions apply.